Mistakes Were Made

Nathan Jones

November 2026

https://lightturtles.github.io

LightTurtles

If your life were a book, what would be the title?

https://lightturtles.github.io

Mudd (2006-2010)



https://lightturtles.github.io

Mudd (2006-2010)



https://lightturtles.github.io

Mudd (2006-2010)

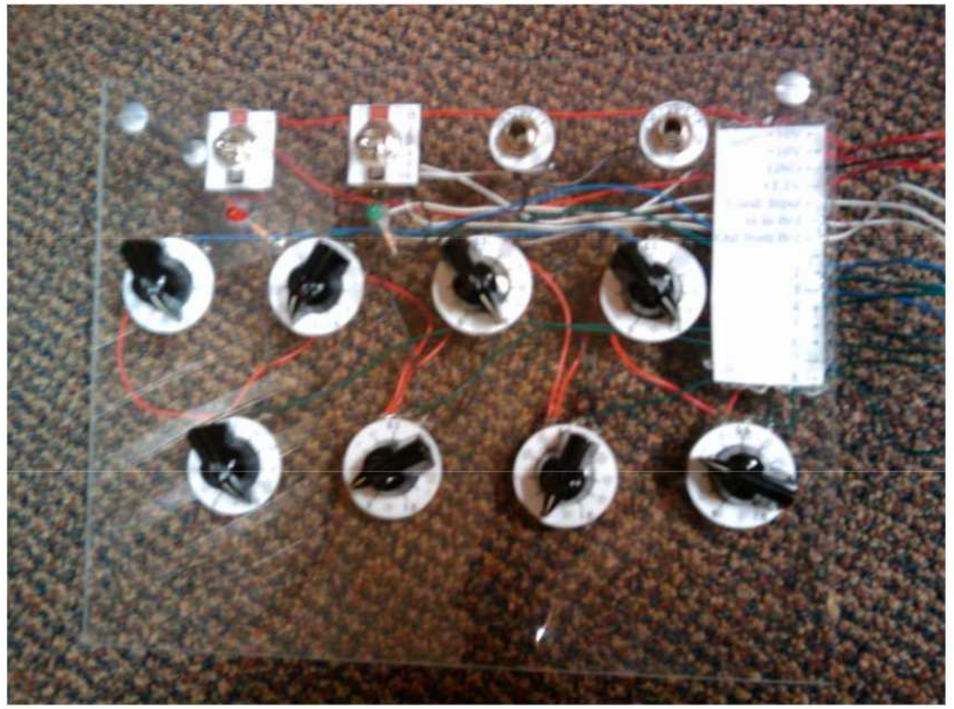


https://lightturtles.github.io

Mudd (2006-2010)

MicroPs (Fall 2008)

The Infinite Impulse Response Filter



https://lightturtles.github.io

Mudd (2006-2010)

MicroPs (Fall 2008)

The Infinite Impulse Response Filter

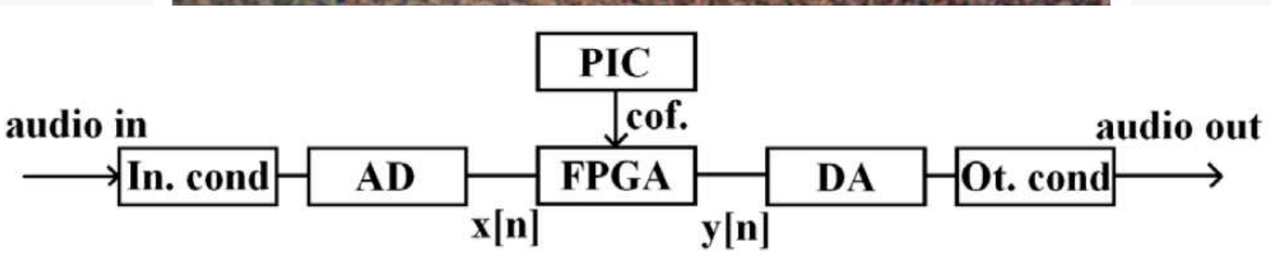


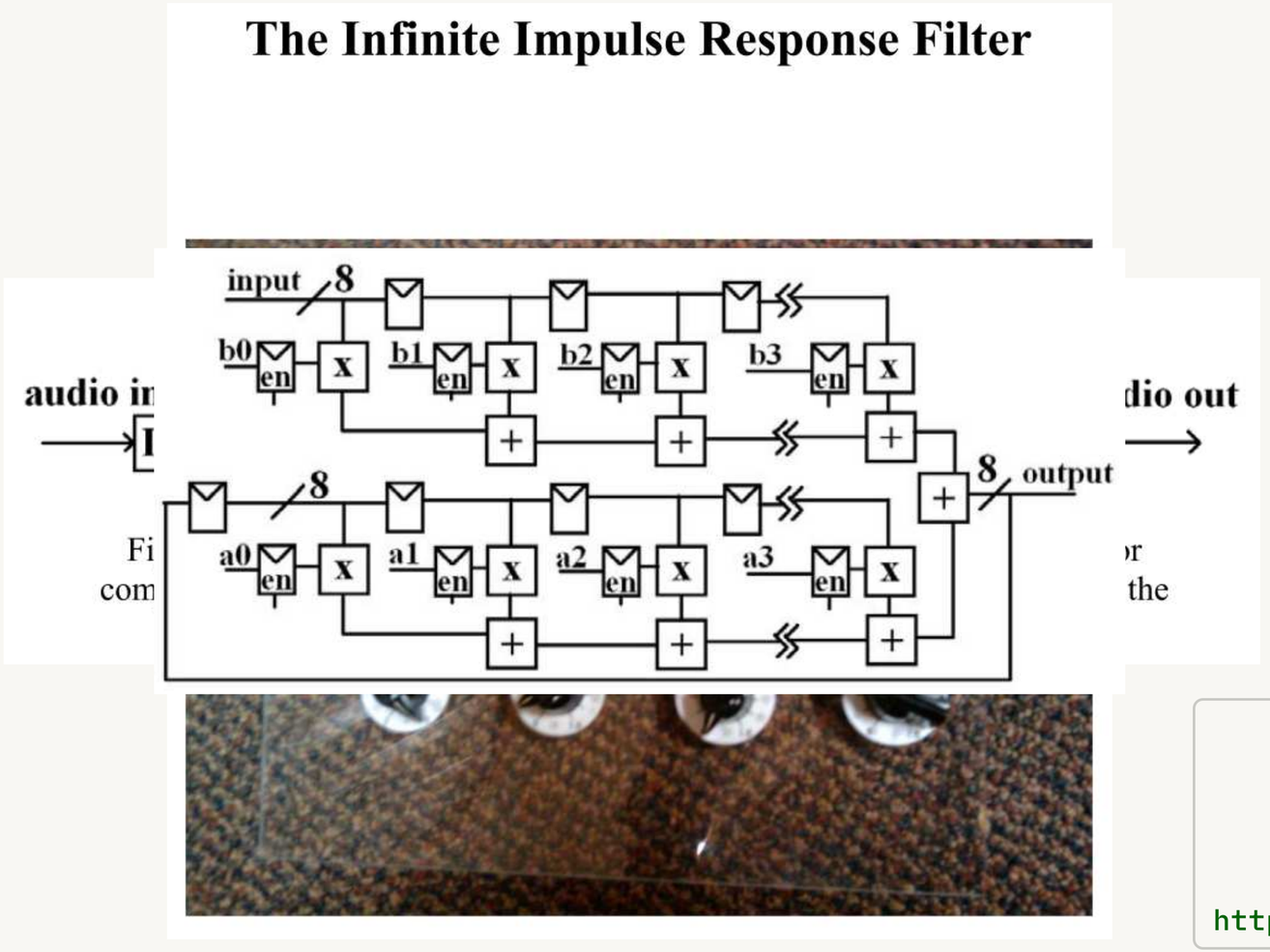
Figure 1: A graphical representation of the IIR filter, including such major components as the input/output conditioning circuitry, the ADC/DAC, and the IIR filter on the PIC/FPGA.



https://lightturtles.github.io

Mudd (2006-2010)

MicroPs (Fall 2008)



https://lightturtles.github.io

Mudd (2006-2010)

MicroPs (Fall 2008)

> “Your ‘infinite impulse response filter’ has seven feedforward taps and only one feedback coefficient. **That impulse response isn’t infinite, it’s barely long enough to finish a sentence.** That’s an FIR filter wearing a fake mustache.”

https://lightturtles.github.io

Mudd (2006-2010)

MicroPs (Fall 2008)

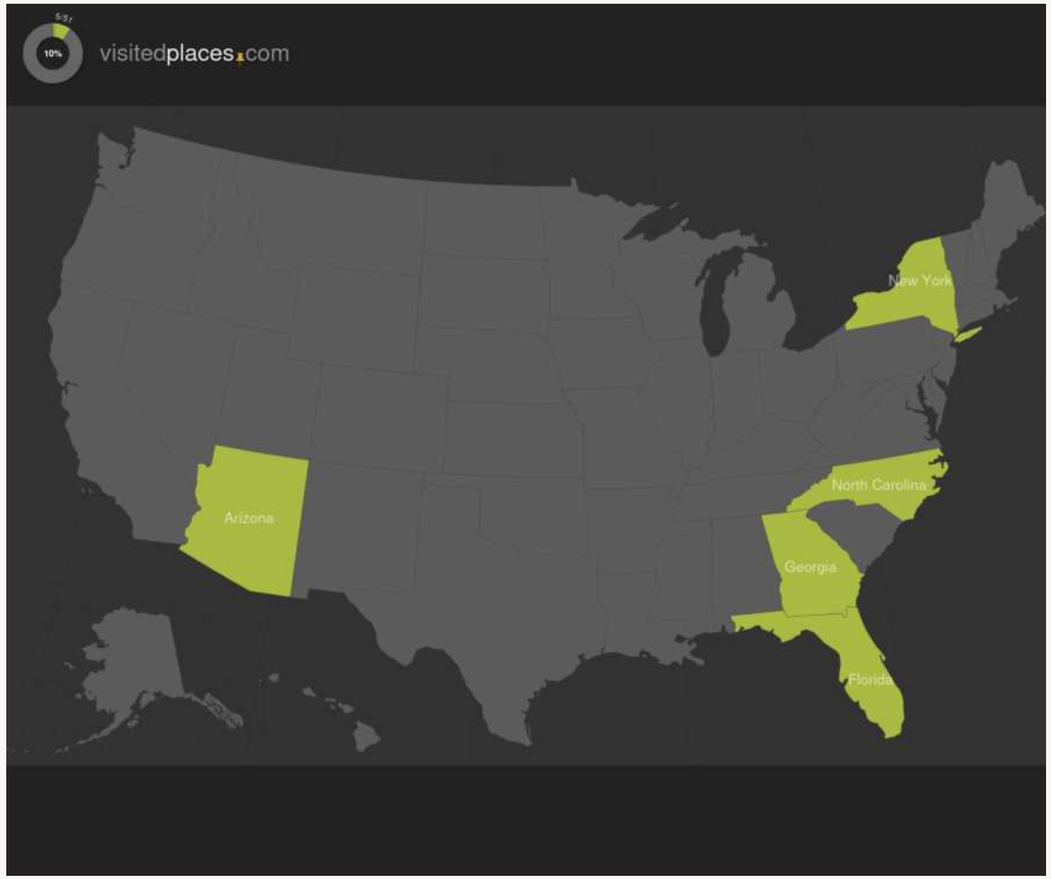
> “Your ‘infinite impulse response filter’ has seven feedforward taps and only one feedback coefficient. **That impulse response isn’t infinite, it’s barely long enough to finish a sentence.** That’s an FIR filter wearing a fake mustache.”

“The signal could only be heard above the noise floor *after* adding a low-pass filter. **You didn’t build an audio filter. You built a noise generator with a guitar input.**”



https://lightturtles.github.io

US Army (2010-2025)





https://lightturtles.github.io

US Army (2010-2025)



https://lightturtles.github.io

US Army (2010-2025)



https://lightturtles.github.io

US Army (2010-2025)



https://lightturtles.github.io

US Army (2010-2025)

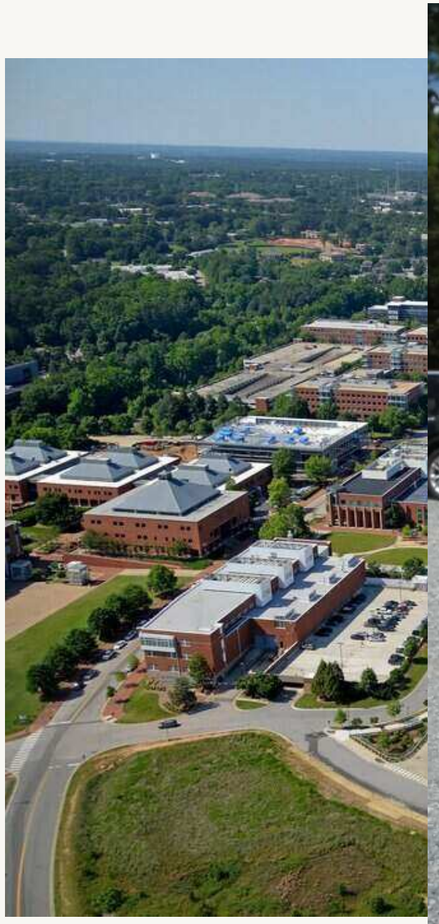
North Carolina State University (2018-2020)



https://lightturtles.github.io

US Army (2010-2025)

North Carolina State University (2018-2020)



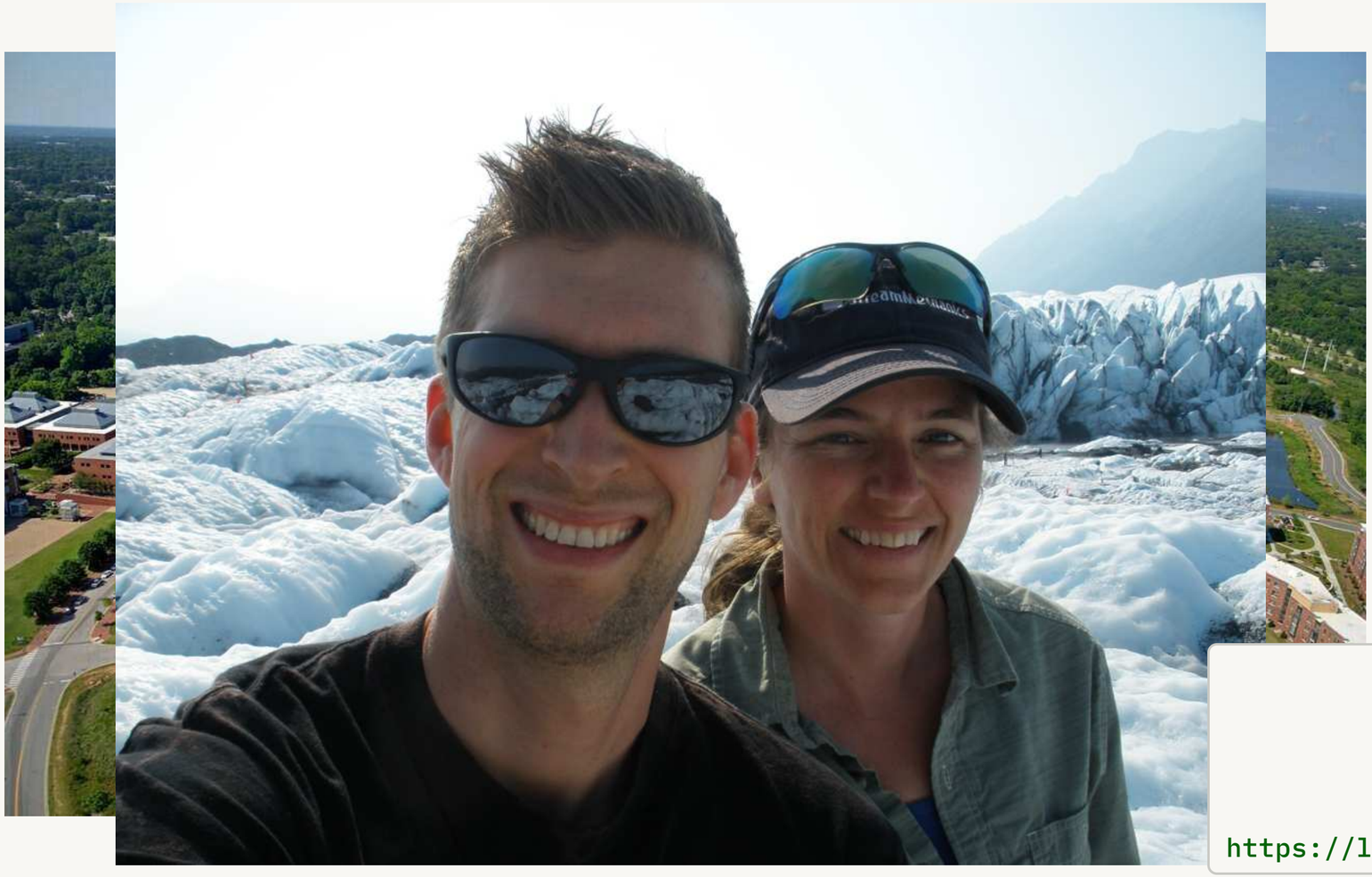




https://lightturtles.github.io

US Army (2010-2025)

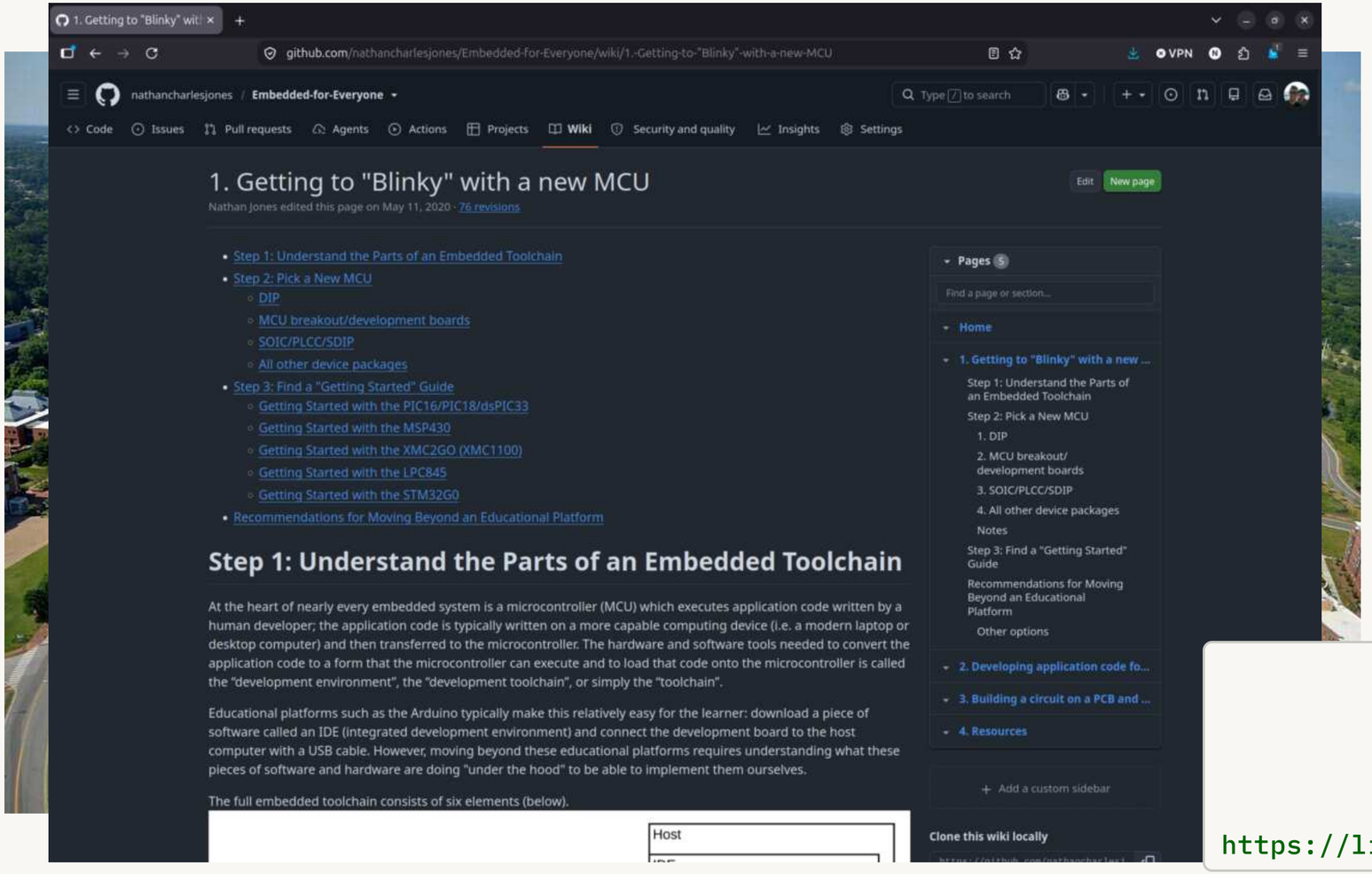
North Carolina State University (2018-2020)



https://lightturtles.github.io

US Army (2010-2025)

North Carolina State University (2018-2020)



https://lightturtles.github.io

US Army (2010-2025)

West Point (2020-2025)



https://lightturtles.github.io

US Army (2010-2025)

West Point (2020-2025)

EE375: Computer Architecture

Home

Unit 0: Intro to EE375

Unit 1: Intro to Computing and the LC-3

Unit 2: Assembly Language Programming

Unit 3: Hardware Support for Input/Output and High-level Languages

Unit 4: Completing the LC-3

Unit 5: Performance Improvements

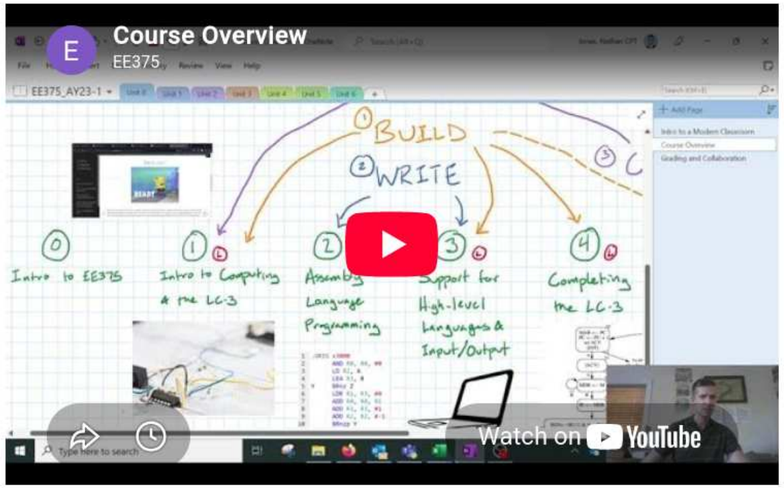
Unit 6: Microcontrollers and Microprocessors

Additional Resources

(MUST DO) Course Overview

Objectives

- Describe the course and unit objectives
- Describe the lab assignments





Additional Resources

- Website for "Introduction to Computing Systems"

https://lightturtles.github.io

US Army (2010-2025)

West Point (2020-2025)

EE375: Computer Architecture

Home

Unit 0: Intro to EE375

Unit 1: Intro to Computing and the LC-3

Unit 2: Assembly Language Programming

Unit 3: Hardware Support for Input/Output and High-level Languages

Unit 4: Completing the LC-3

Unit 5: Performance Improvements

Unit 6: Microcontrollers and Microprocessors

Additional Resources





https://lightturtles.github.io

US Army (2010-2025)

West Point (2020-2025)

EE375: Computer Architecture

Home

Unit 0: Intro to EE375

Unit 1: Intro to Computing and the LC-3

Unit 2: Assembly Language Programming

Unit 3: Hardware Support for Input/Output and High-level Languages

Unit 4: Completing the LC-3

Unit 5: Performance Improvements

Unit 6: Microcontrollers and Microprocessors

Additional Resources





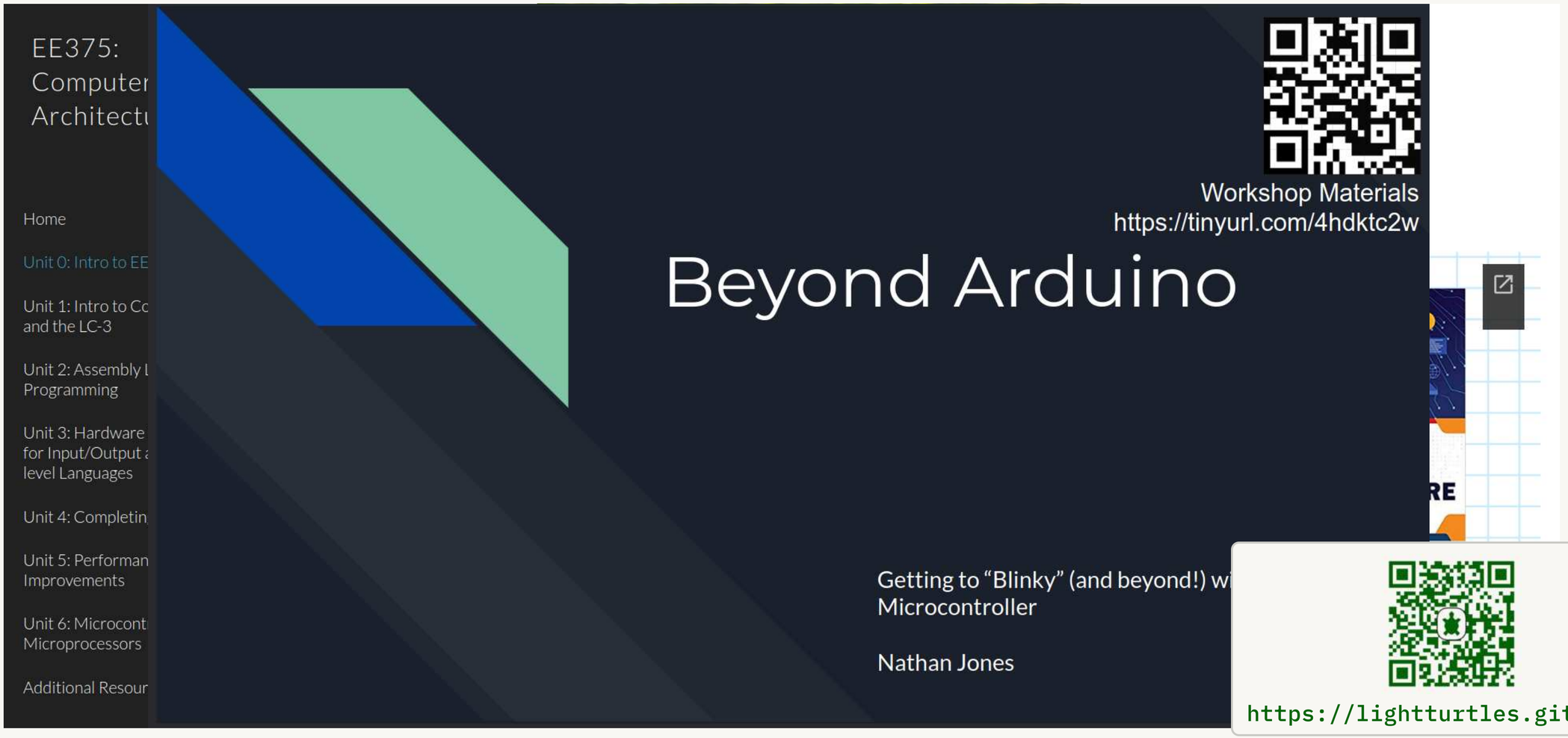




https://lightturtles.github.io

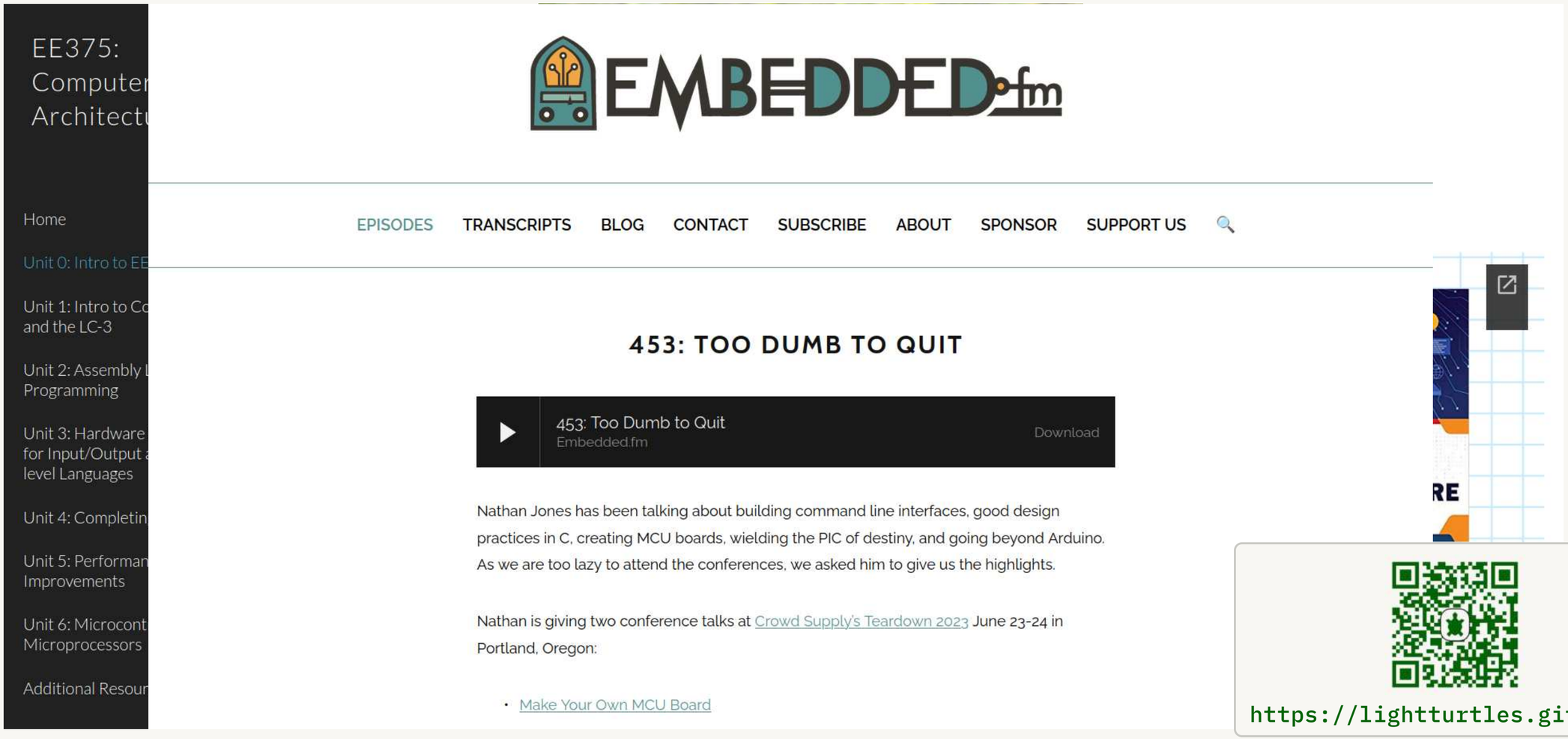
US Army (2010-2025)

West Point (2020-2025)



US Army (2010-2025)

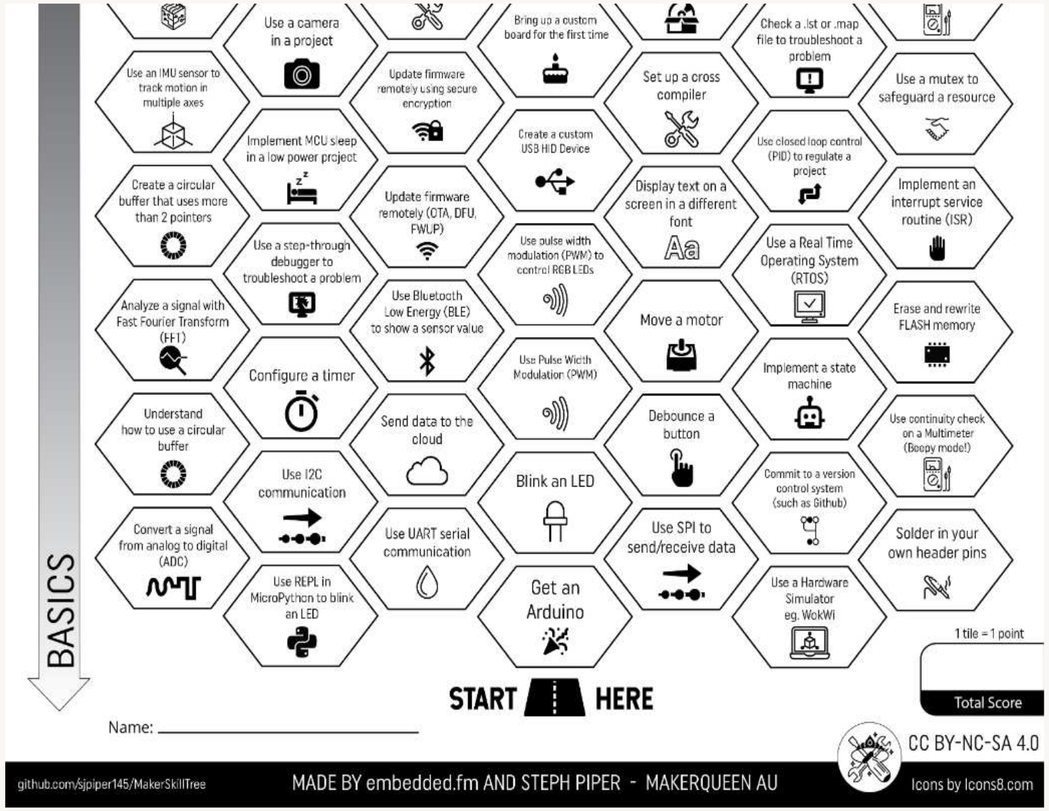
West Point (2020-2025)

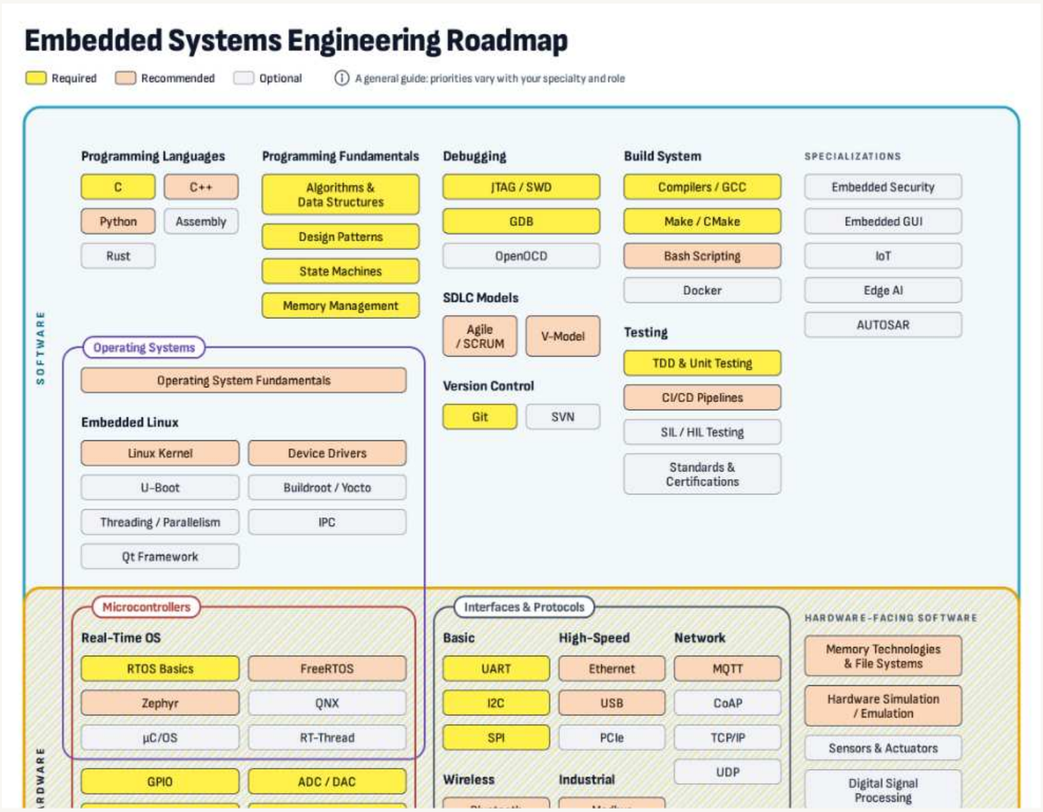


https://lightturtles.github.io

Embedded Systems Roadmaps

Article on EmbeddedRelated.com

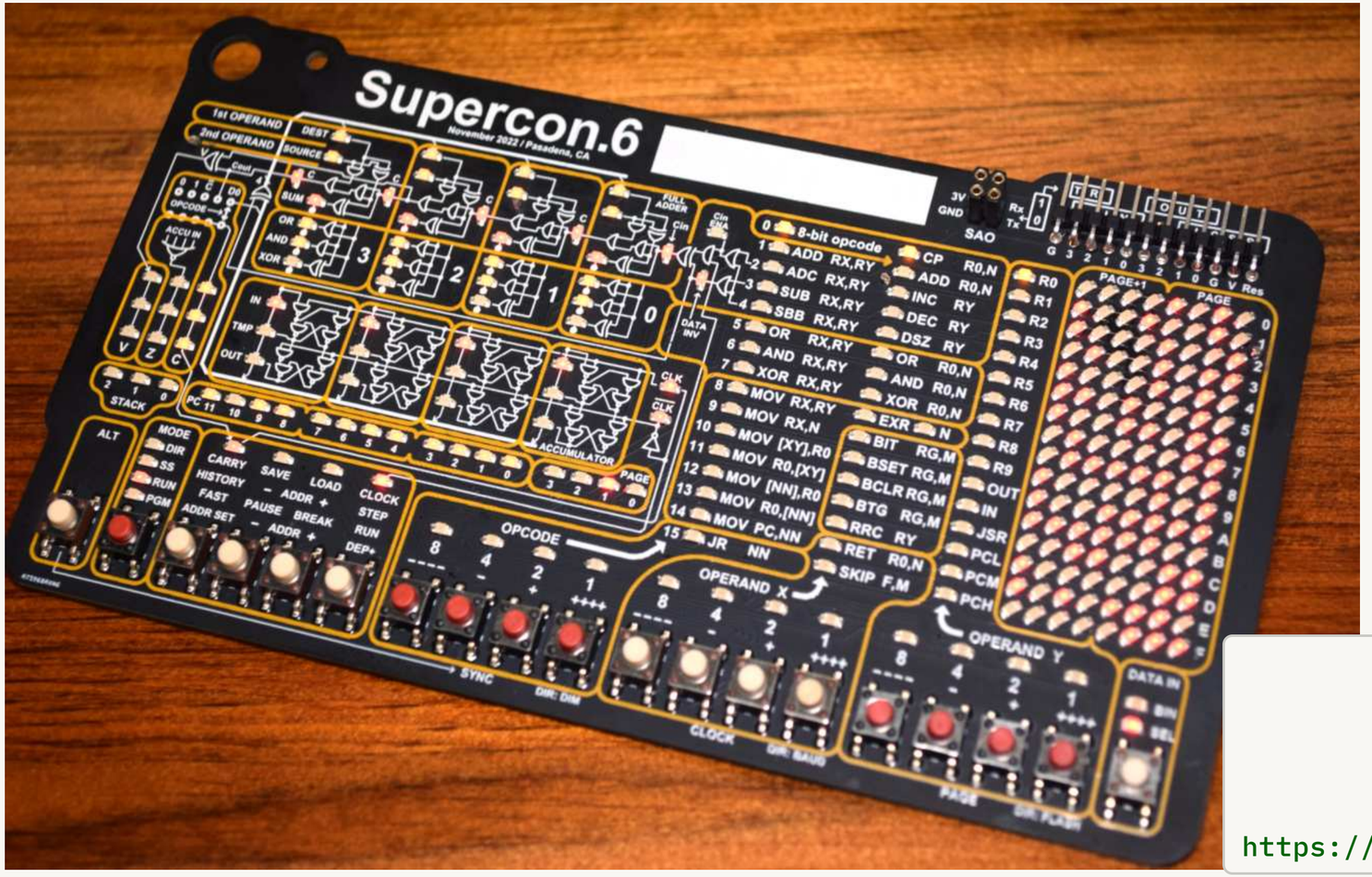




https://lightturtles.github.io

Inside the Voja4

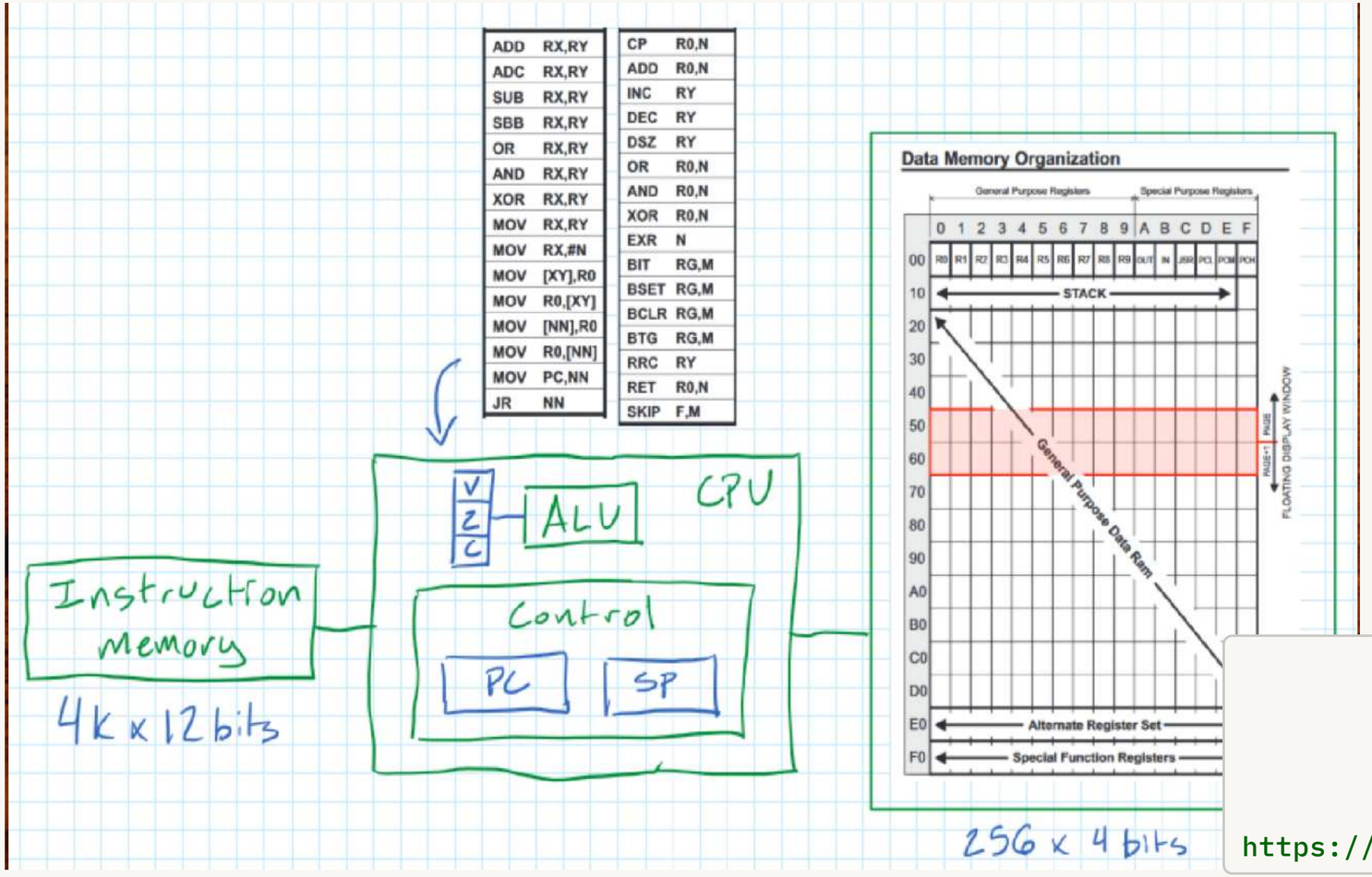
Talk at Hackaday Supercon



https://lightturtles.github.io

Inside the Voja4

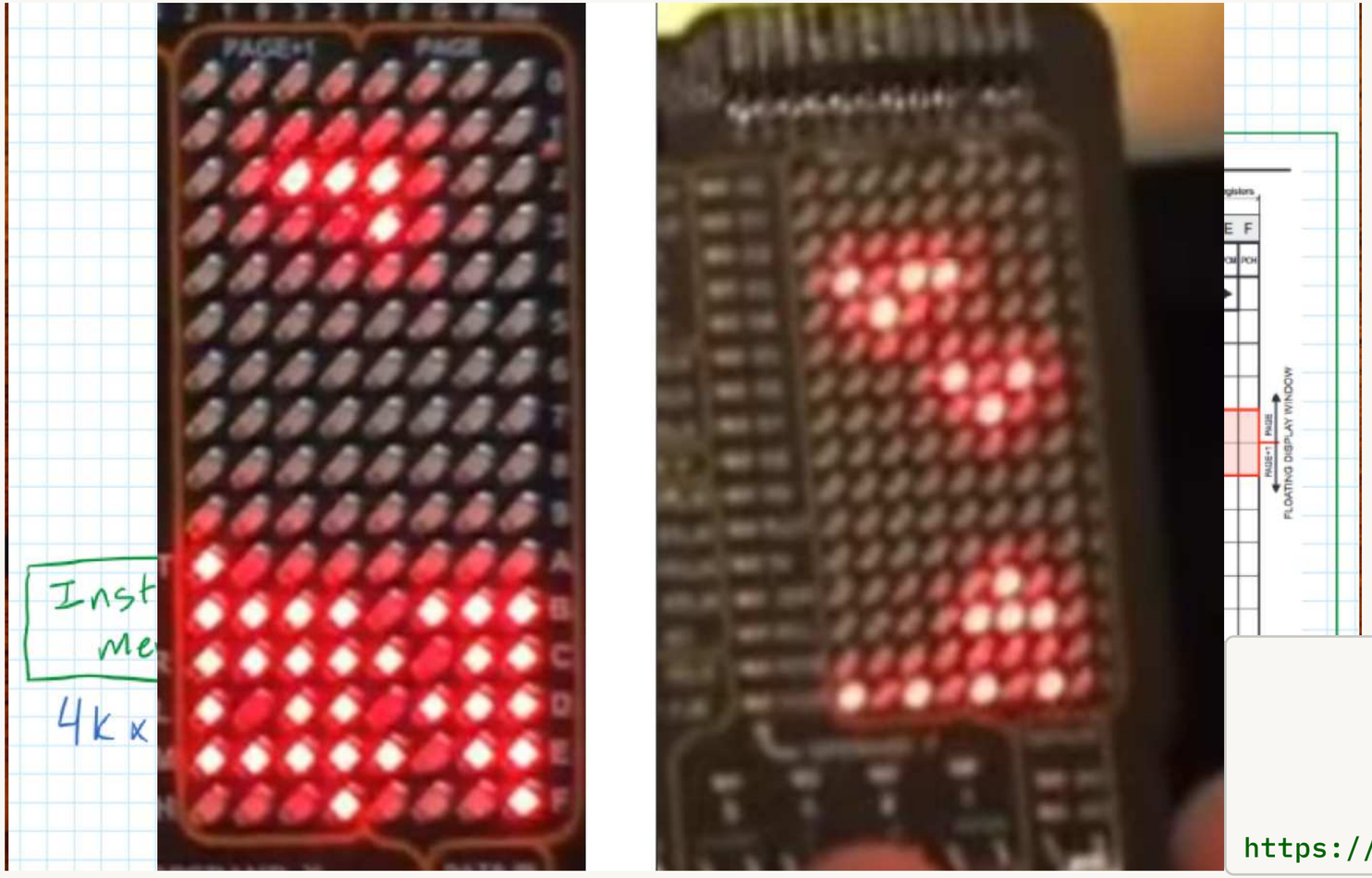
Talk at Hackaday Supercon



https://lightturtles.github.io

Inside the Voja4

Talk at Hackaday Supercon



https://lightturtles.github.io

What does it mean to be “Turing Complete”?

Article on EmbeddedRelated.com

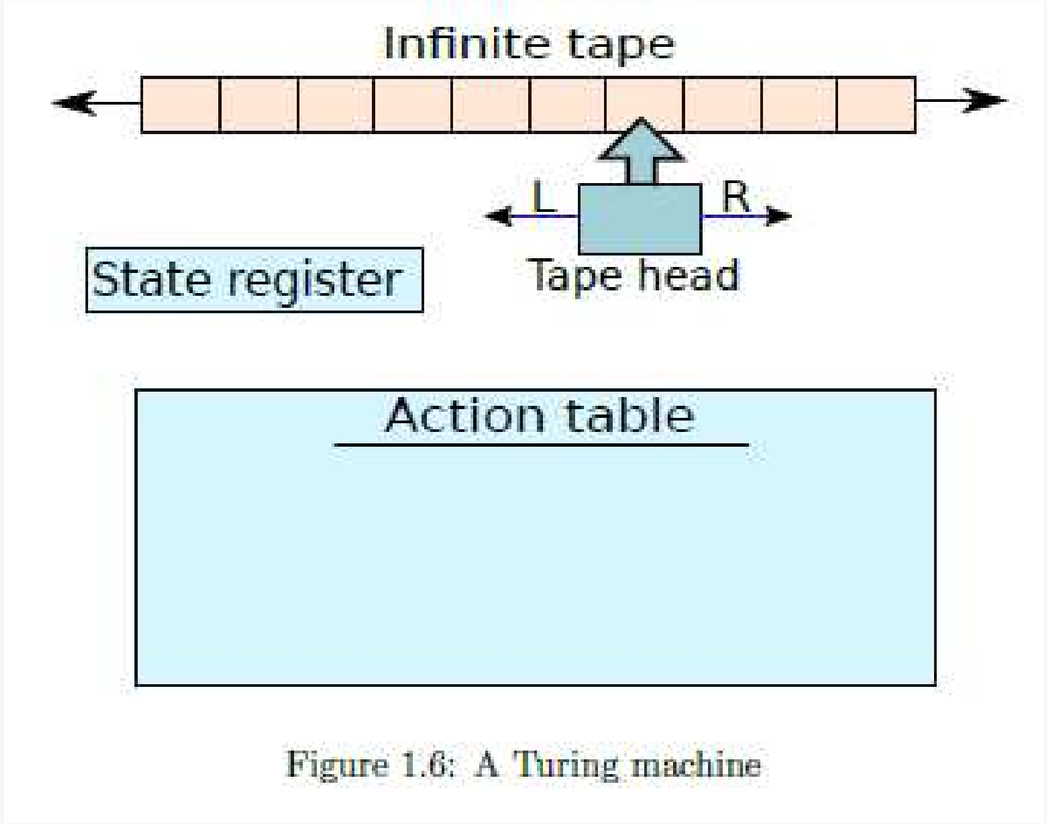


Figure 1.6: A Turing machine

$f: \mathbb{N}^k \rightarrow \mathbb{N}$

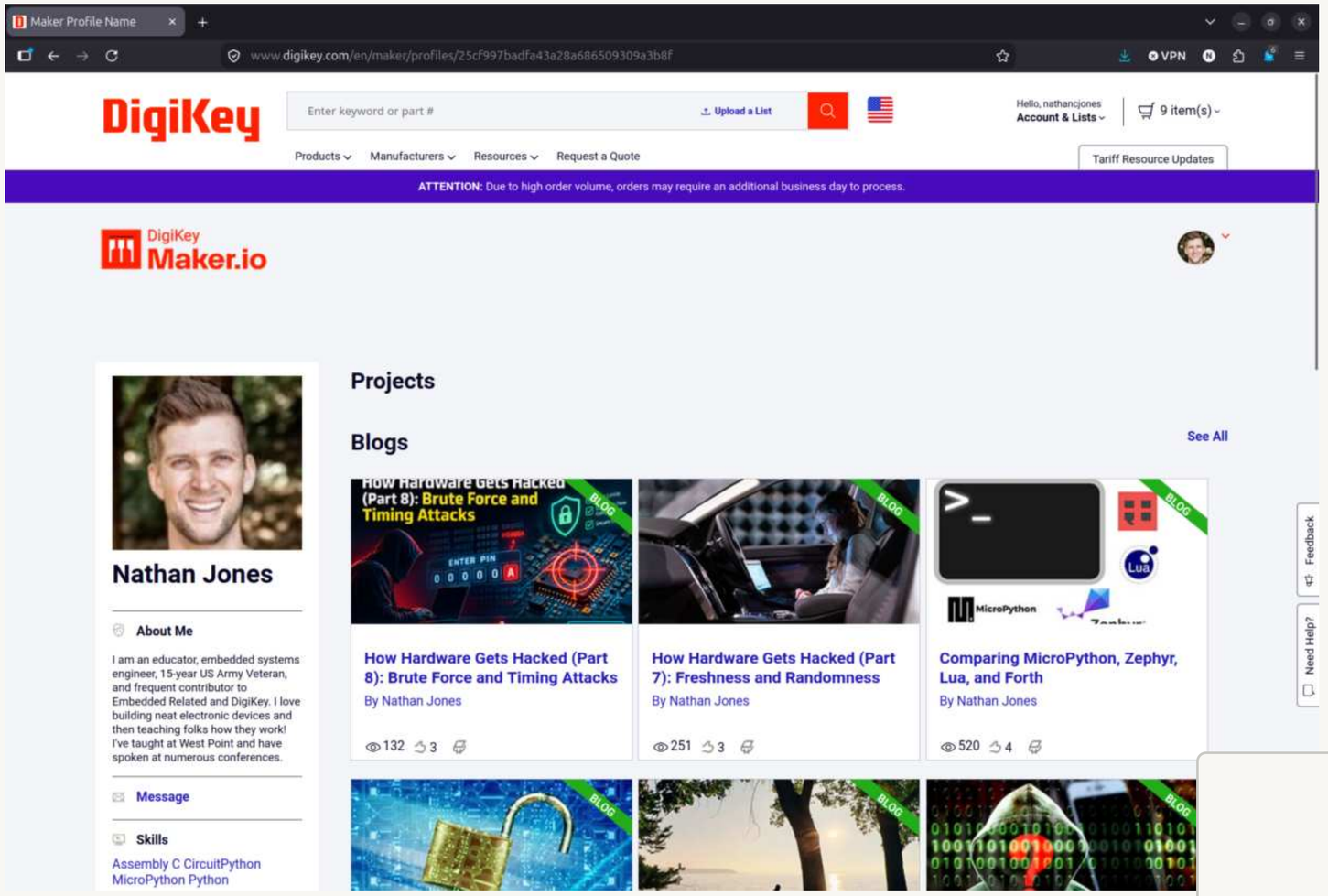
Computable (by a Turing machine)
i.e. Turing-complete

- Intel Core i7
- ATMega328
- etc

Not-computable
e.g. Halting problem

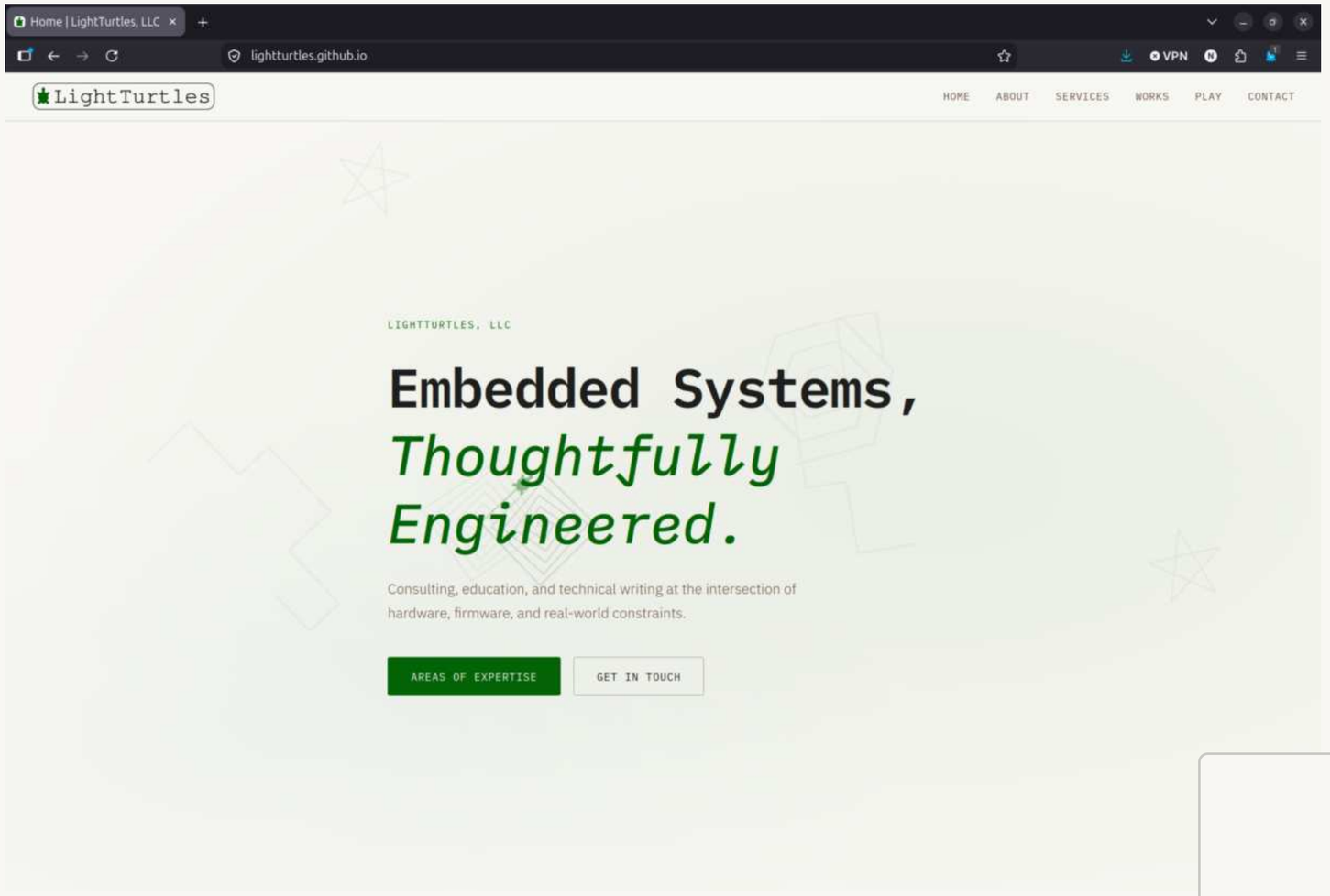
https://lightturtles.github.io

LightTurtles, LLC (2025-Pres)



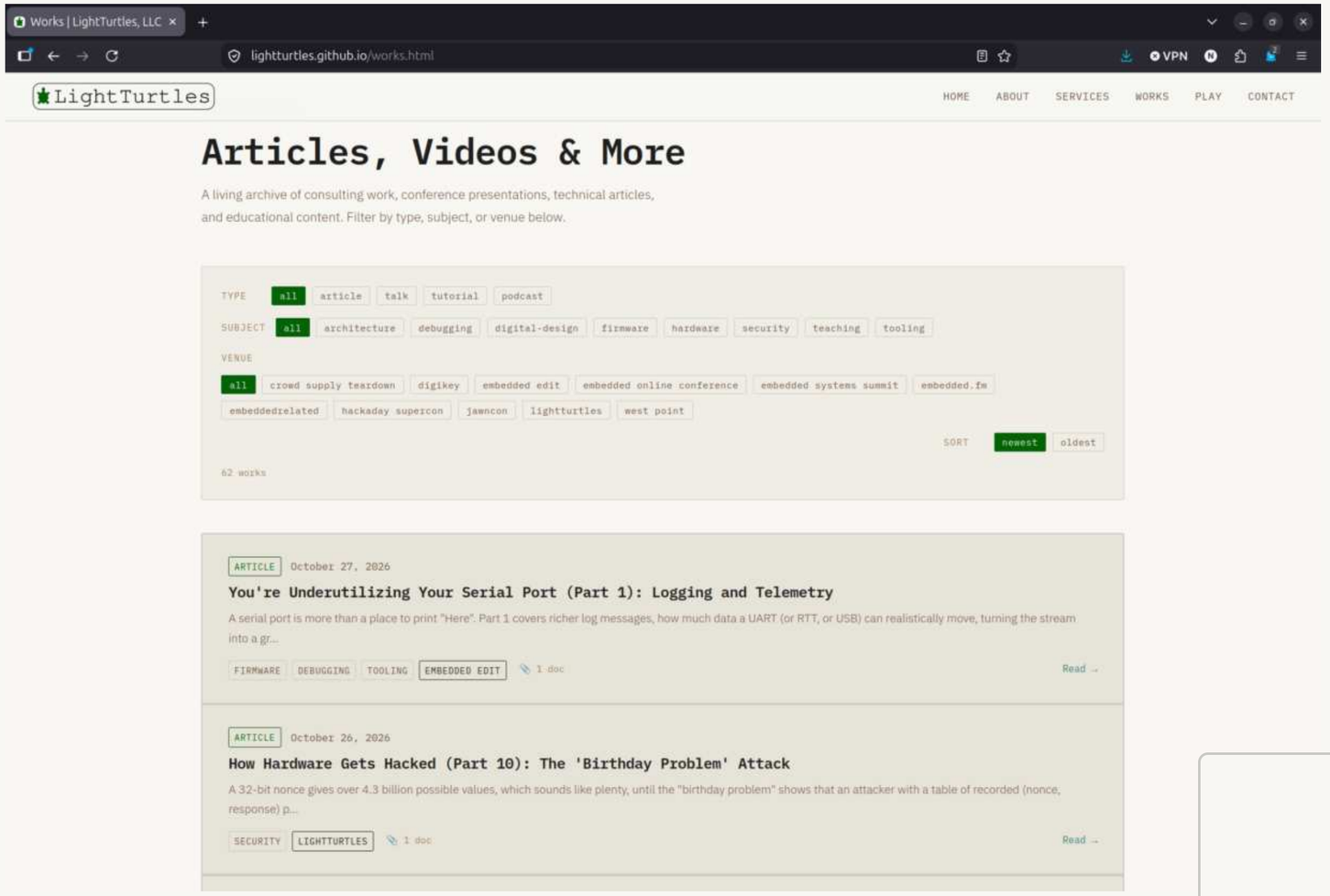
https://lightturtles.github.io

LightTurtles, LLC (2025-Pres)



https://lightturtles.github.io

LightTurtles, LLC (2025-Pres)

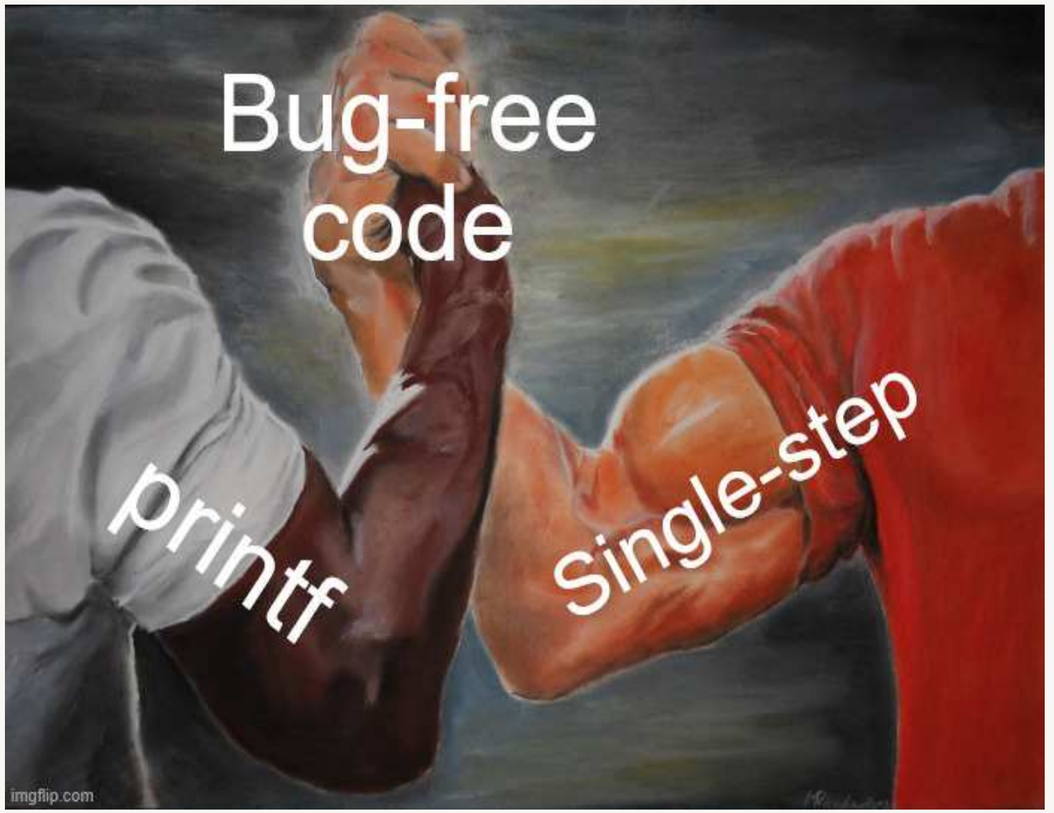


https://lightturtles.github.io

In Defense of `printf` Debugging

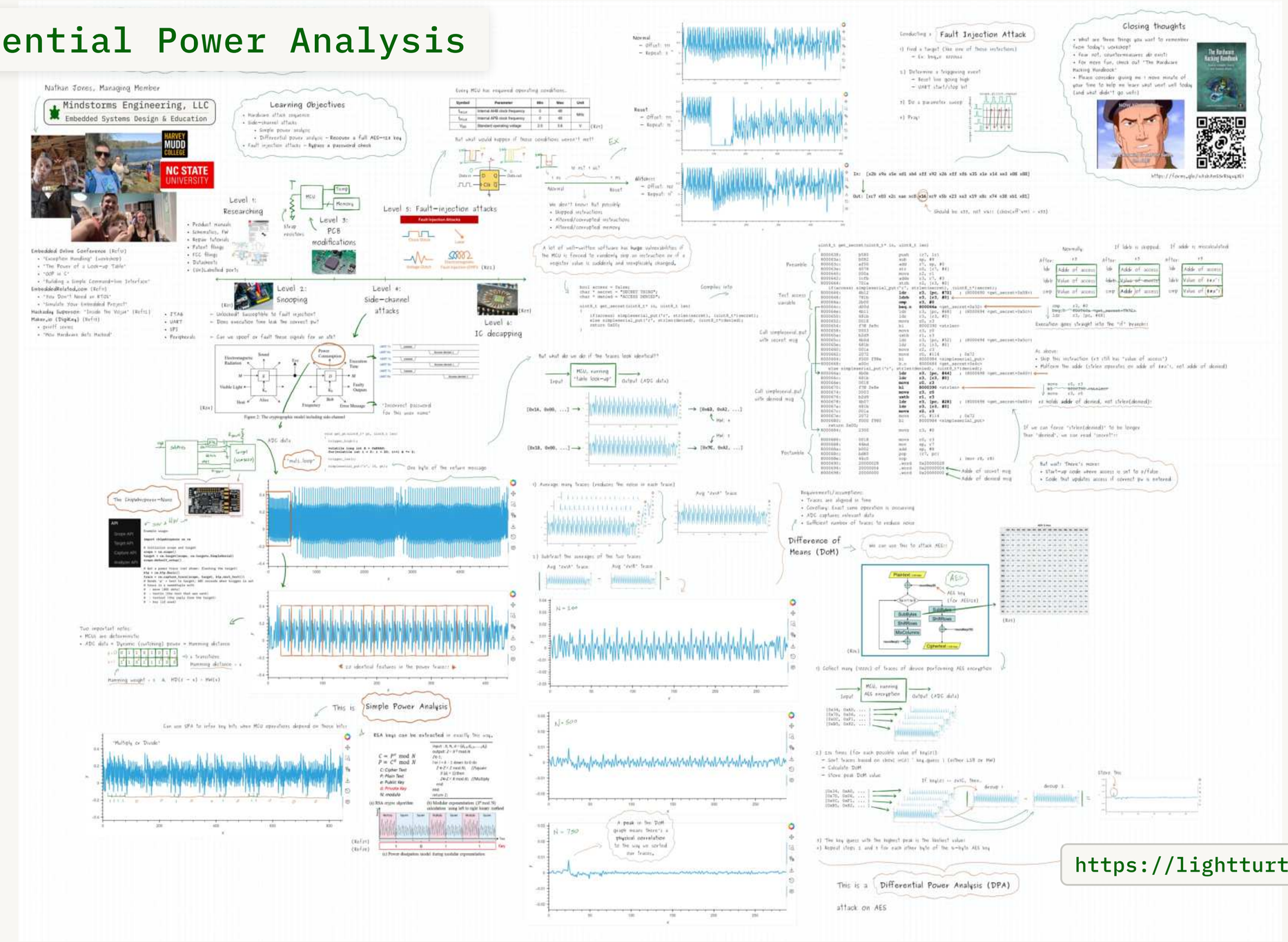
Article series on DigiKey (9 total)

```
[I] A.c[2]  read_string() A enqueueing data
[I] A.c[2]  read_string() A enqueueing data
[I] D.c[45] dequeue()     Dequeuing data
[I] B.c[9]  get_msg()     B enqueueing data
[I] A.c[2]  read_string() A enqueueing data
[I] A.c[2]  read_string() A enqueueing data
[I] D.c[45] dequeue()     Dequeuing data
[I] A.c[2]  read_string() A enqueueing data
[I] A.c[2]  read_string() A enqueueing data
[W] D.c[75] enqueue()     Queue at 75% capacity
[I] A.c[2]  read_string() A enqueueing data
[I] A.c[2]  read_string() A enqueueing data
[I] C.c[21] read_ADC()    C enqueueing data
[E] D.c[78] enqueue()     Queue at max capacity;
                          unable to proceed
...System reset...
```



https://lightturtles.github.io

Differential Power Analysis



Differential Power Analysis

Voltage Glitch

Fault Injection (EMFI) (R02)

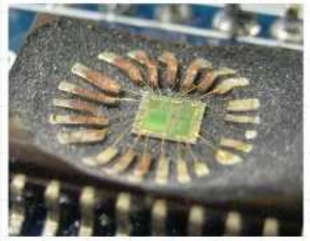




(R01)

Level 2:
Snooping

Level 4:
Side-channel
attacks



(R03)

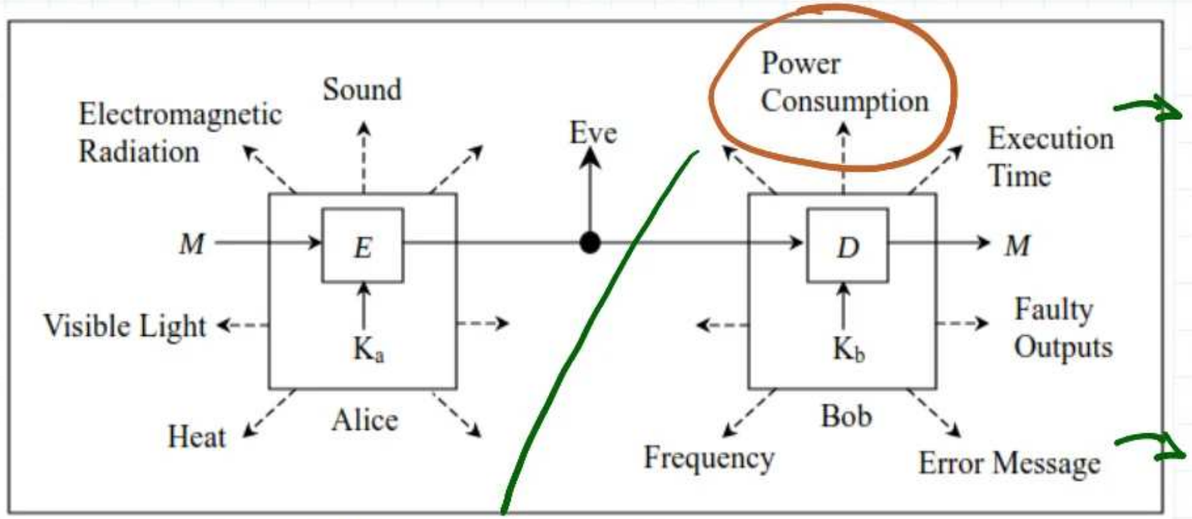
Level 6:
IC decapping

TAG ← Unlocked? Susceptible to fault injection?

ART ← Does execution time leak the correct pw?

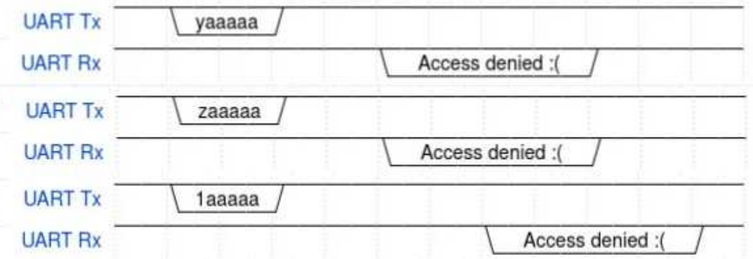
PI

eripherals ← Can we spoof or fault these signals for an atk?



(R04)

Figure 2: The cryptographic model including side-channel



"Incorrect password for this user name"

```
void get_pt(uint8_t* pt, uint8_t len)
{
  trigger_high();

  volatile long int A = 0xE6A9;
  for(volatile int i = 0; i < 20; i++) A *= 2;

  trigger_low();
```

"mul2_loop"

Devices leak secrets in many ways.

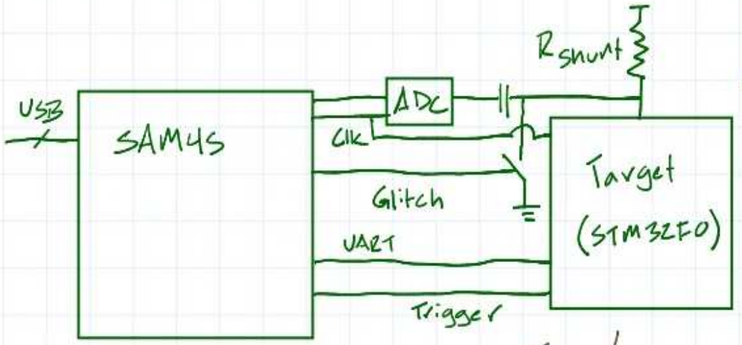
https://lightturtles.github.io

Differential Power Analysis



"Incorrect password for this user name"

Figure 2: The cryptographic model including side-channel



ADC data

"mul2_loop"

```
void get_pt(uint8_t* pt, uint8_t len)
{
  trigger_high();

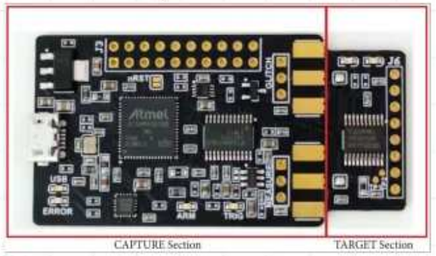
  volatile long int A = 0xE6A9;
  for(volatile int i = 0; i < 20; i++) A *= 2;

  trigger_low();

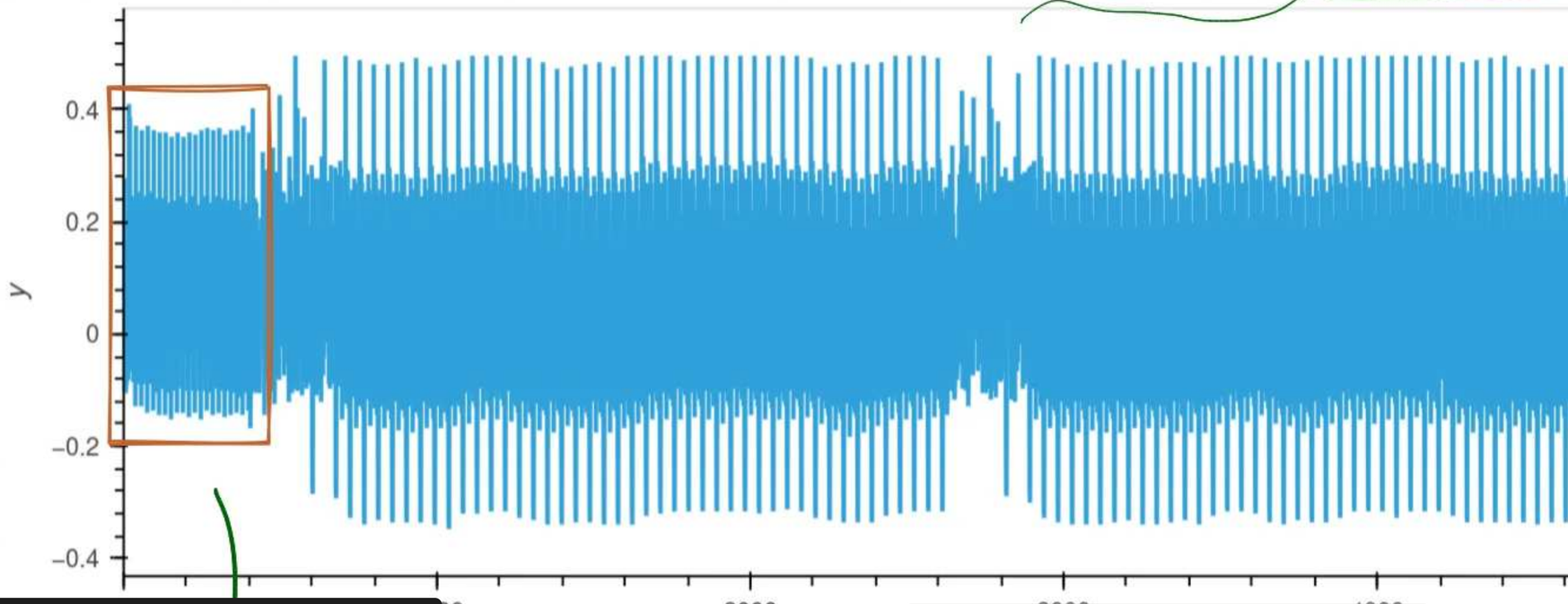
  simpleserial_put('r', 16, pt);
}
```

One byte of the return message

Whisperer-Nano



SW & HW




```
Example usage:

import chipwhisperer as cw

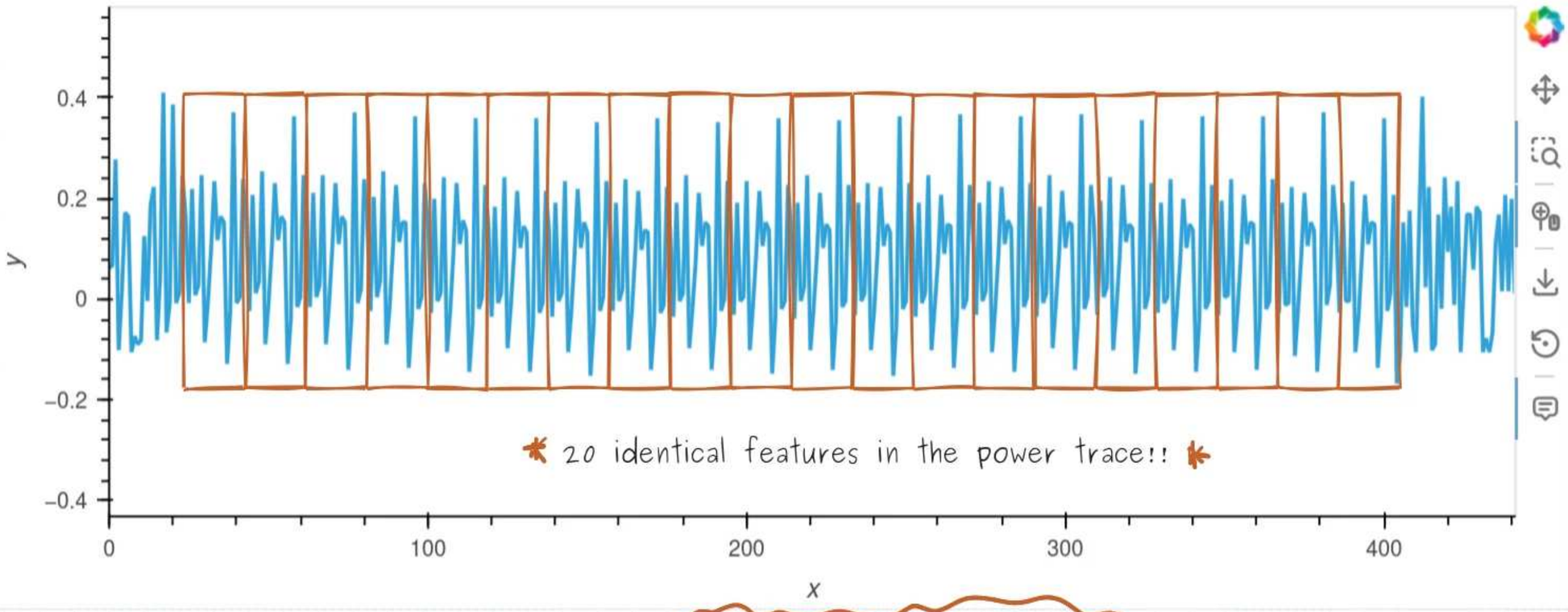
# Initialize scope and target
scope = cw.scope()
target = cw.target(scope, cw.targets.SimpleSerial)
scope.default_setup()

# Get a power trace (not shown: flashing the target)
#   - wave (ADC data)
#   - textin (the text that was sent)
```

Measurement setup using the ChipWhisperer-Nano

https://lightturtles.github.io

Differential Power Analysis



This is Simple Power Analysis

those bits!

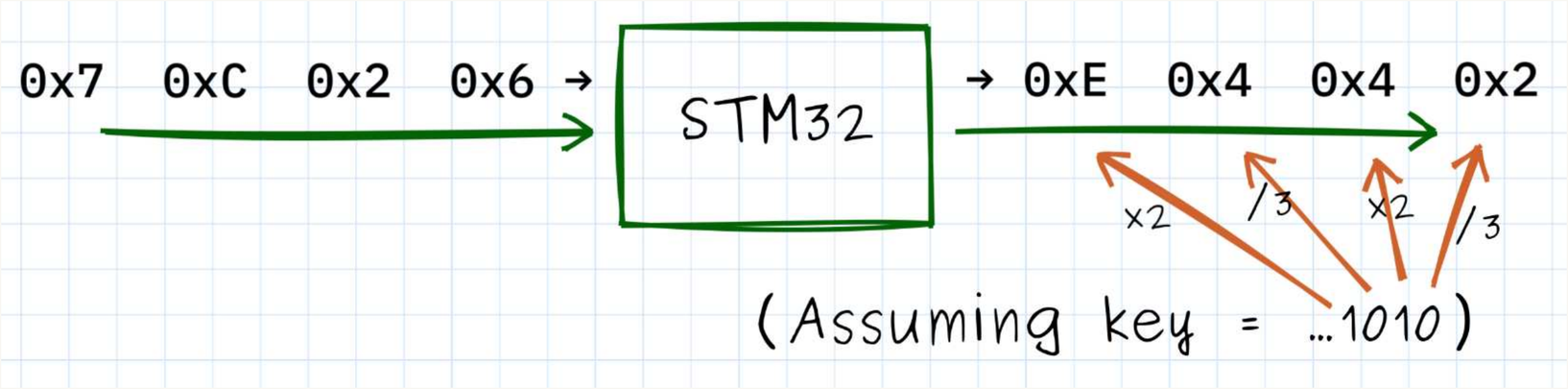
RSA keys can be extracted in exactly this way.

input : $X, N, d = (d_{k-1}, d_{k-2}, \ldots\ldots, d_0)$

Simple Power Analysis (SPA)

https://lightturtles.github.io

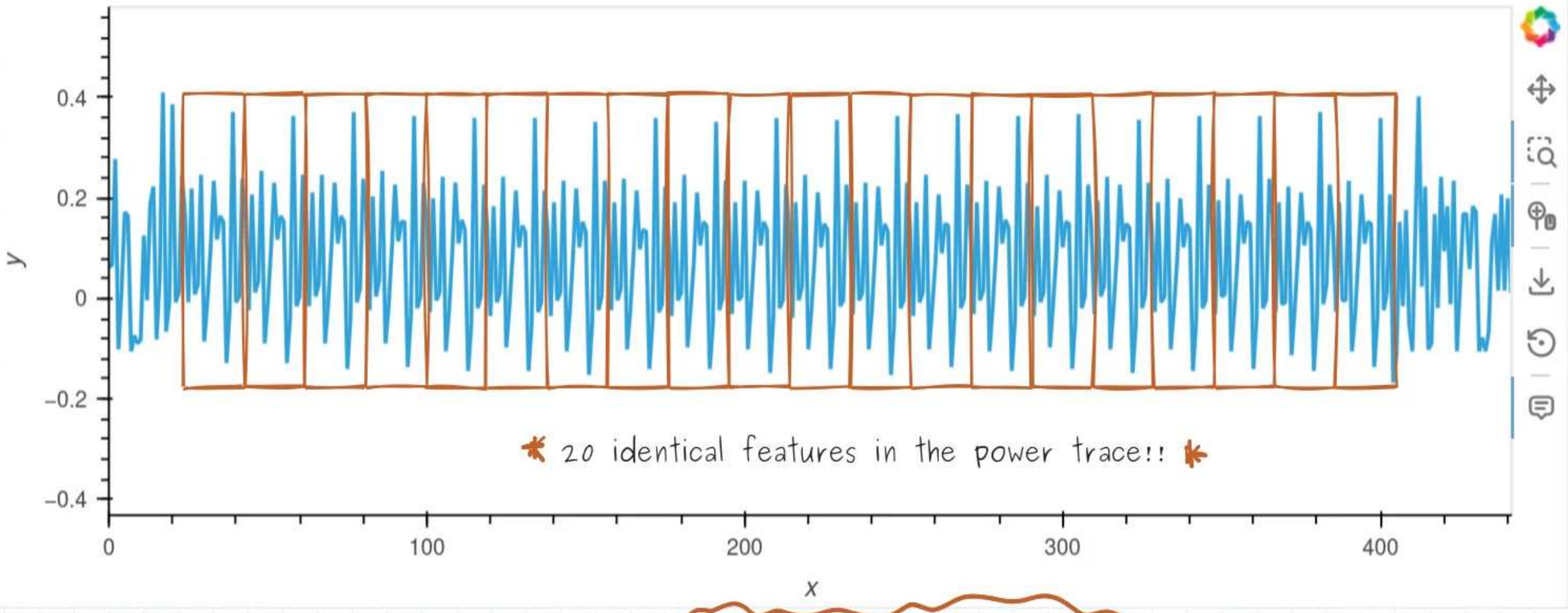
“Multiply or Divide”




```
uint16_t key = /* SECRET */;
for(size_t idx = 0; idx < 16; idx++)
{
    if(key & (1<<idx)) pt[idx] = pt[idx]*2;
    else               pt[idx] = pt[idx]/3;
}
```

https://lightturtles.github.io

Differential Power Analysis



This is **Simple Power Analysis**

er key bits when MCU operations depend on those bits!

RSA keys can be extracted in exactly this way.

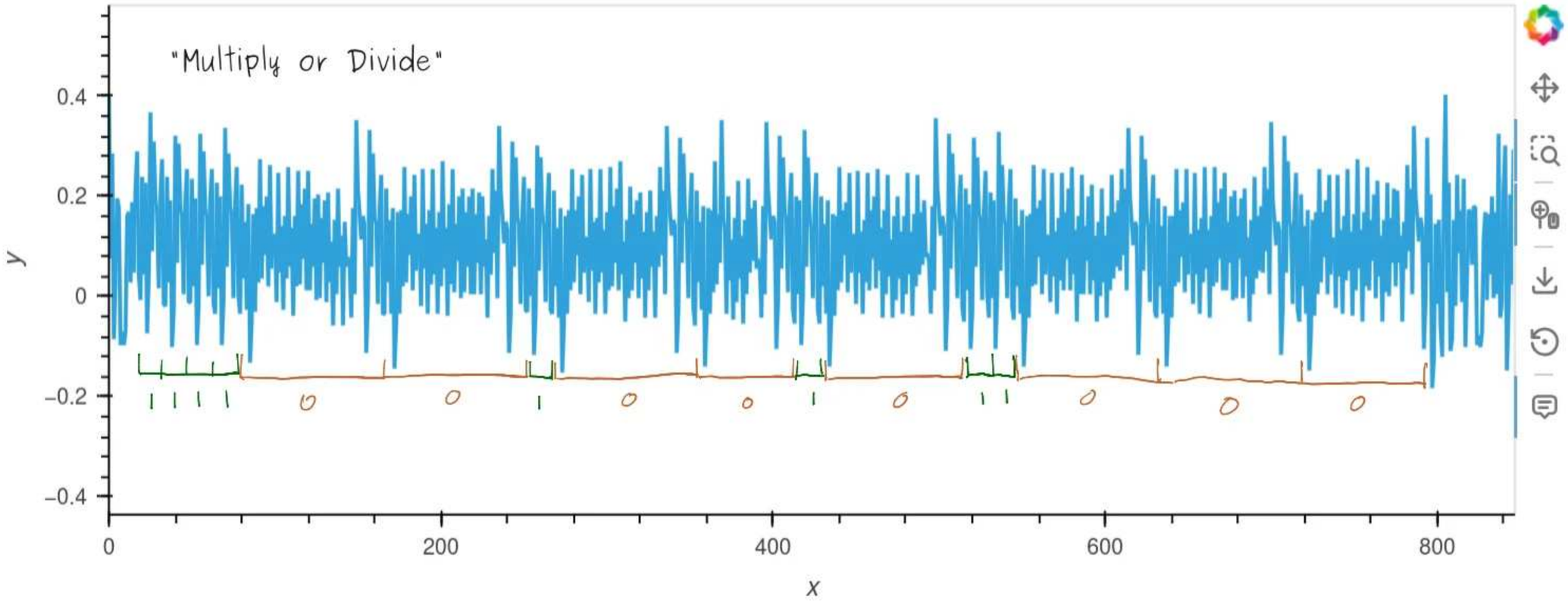
input : $X, N, d = (d_{k-1}, d_{k-2}, \ldots\ldots, d_0)$

https://lightturtles.github.io

Differential Power Analysis

This is Simple

Can use SPA to infer key bits when MCU operations depend on those bits!



RSA

$C =$
$P =$

C: C
P: P
e: P
d: P
N: n

(a) RSA cr



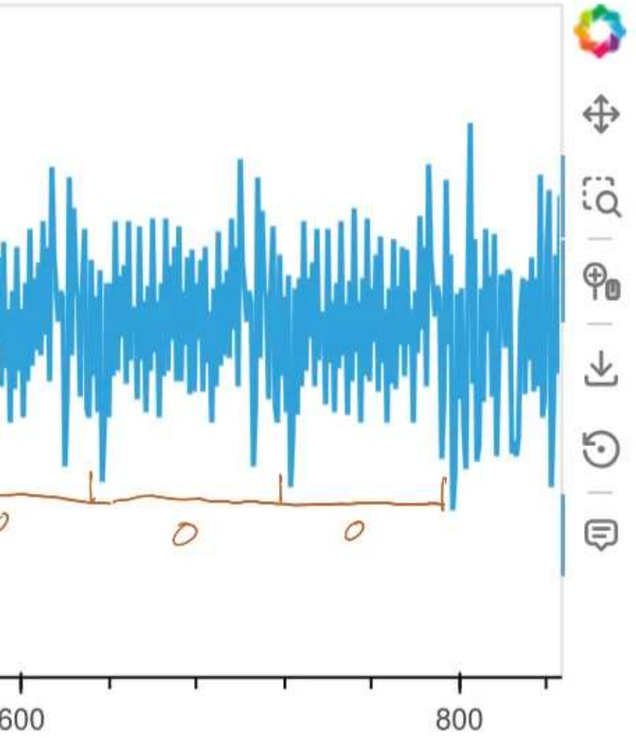
(Refor)

(c)

We can determine the bits of the secret key by identifying multiply and divide operations in the power trace!

https://lightturtles.github.io

Differential Power Analysis



RSA keys can be extracted in exactly this way.

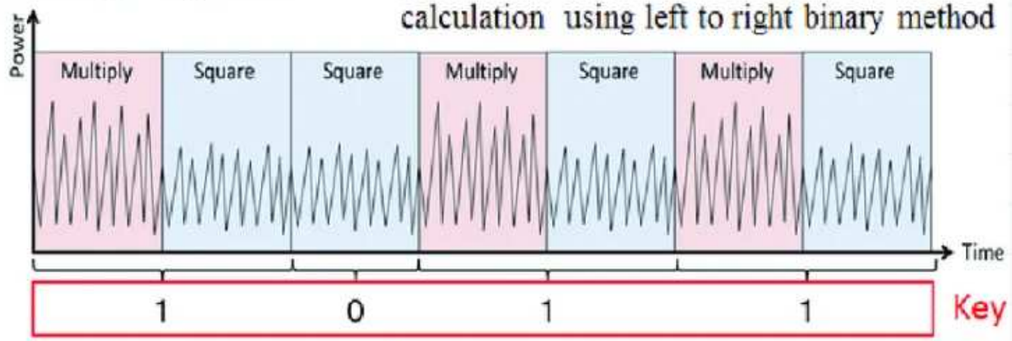
$$C = P^e \bmod N$$
$$P = C^d \bmod N$$

C: Cipher Text
P: Plain Text
e: Public Key
d: Private Key
N: modulo

(a) RSA crypto algorithm

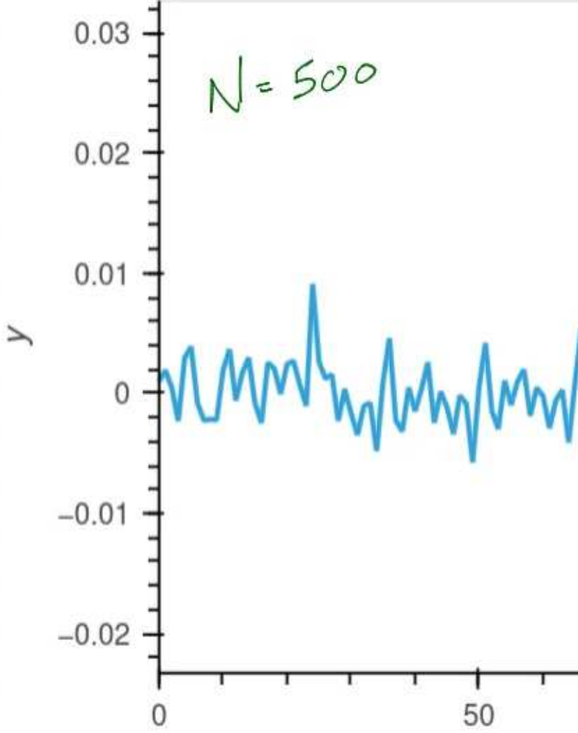
```
input : X, N, d = (d_{k-1}, d_{k-2}, ....., d_0)
output: Z = X^d mod N
Z←1;
For i = k − 1 down to 0 do
  Z ←Z × Z mod N;     //Square
    if (d_i = 1) then
    Z←Z × X mod N;  //Multiply
  end
end
return Z;
```

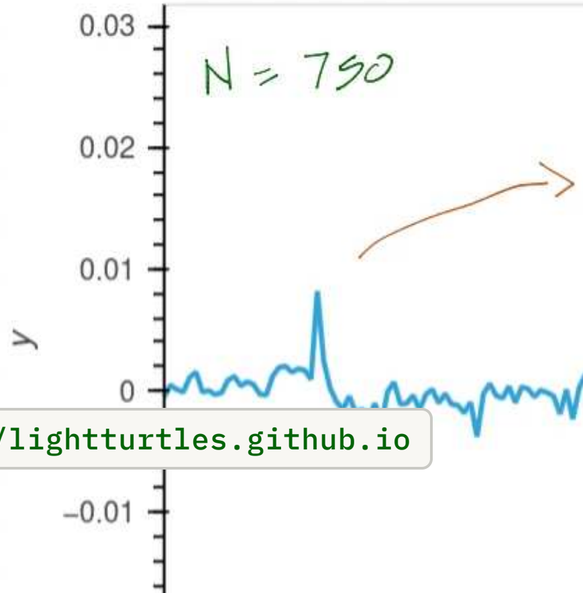
(b) Modular exponentiation (X^d mod N) calculation using left to right binary method



(c) Power dissipation model during modular exponentiation

(Ref07)
(Ref08)





This is exactly how RSA is broken using SPA.

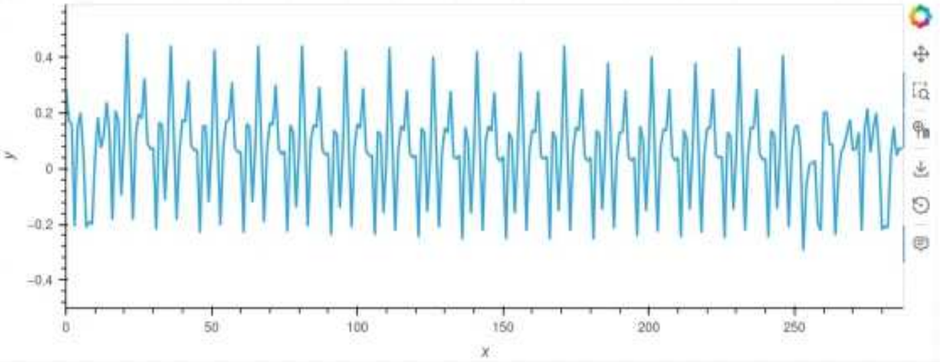
https://lightturtles.github.io

Differential Power Analysis

But what do we do if the traces look identical??

Input → MCU, running "table look-up" → Output (ADC data)

[0x1A, 0x00, ...] →



→ [0x63, 0xA2, ...]

HW: 4

HW: 5

[0x1B, 0x00, ...] →



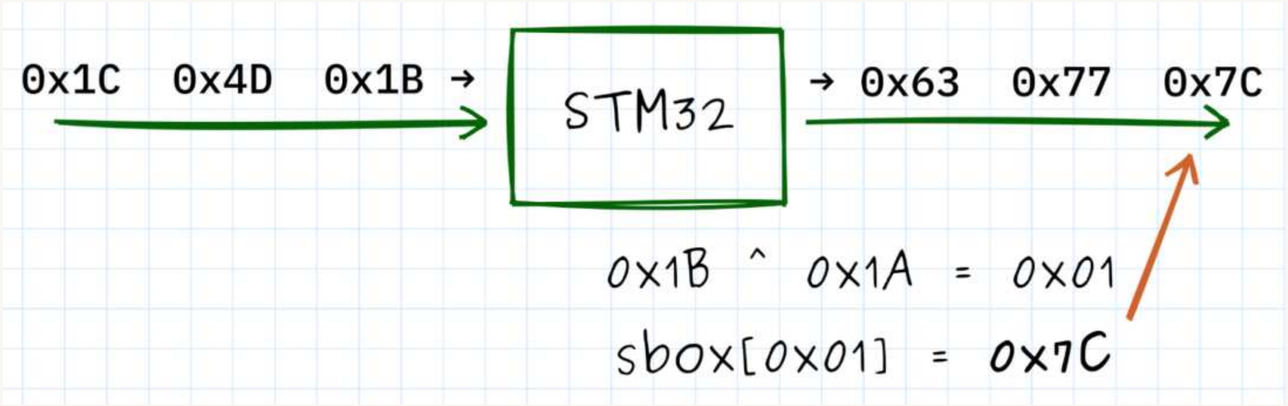
→ [0x7C, 0xA2, ...]

Power analysis can even help us determine what data is being operated on.

https://lightturtles.github.io

1) Average many traces (reduces the noise in each trace)

“Table Look-up”




```c
static const uint8_t sbox[256] = {0x63, 0x7c, 0x77, /* ... */};
uint8_t                key[16] = {0x1a, 0x4f, 0x1c, /* ... */};

for(size_t idx = 0; idx < 16; idx++)
{
    pt[idx] = sbox[ pt[idx]^key[idx] ];
}
```

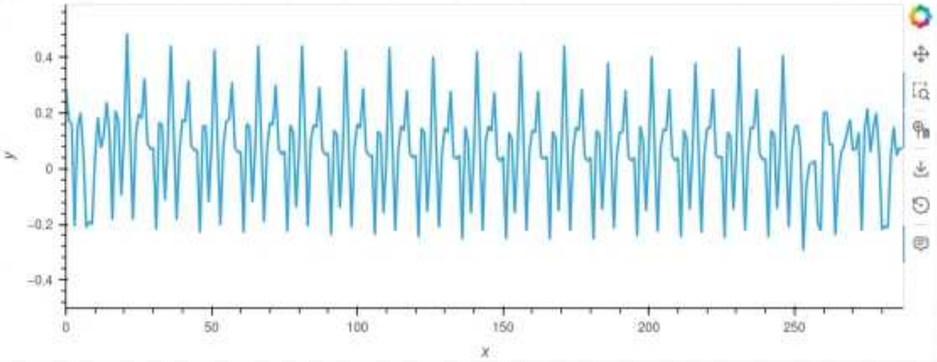
https://lightturtles.github.io

Differential Power Analysis

But what do we do if the traces look identical??



`[0x1A, 0x00, ...]` →

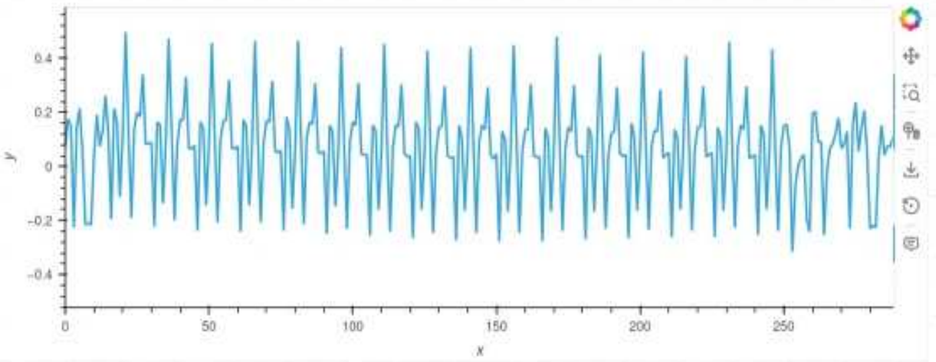


→ `[0x63, 0xA2, ...]`

HW: 4

HW: 5

`[0x1B, 0x00, ...]` →



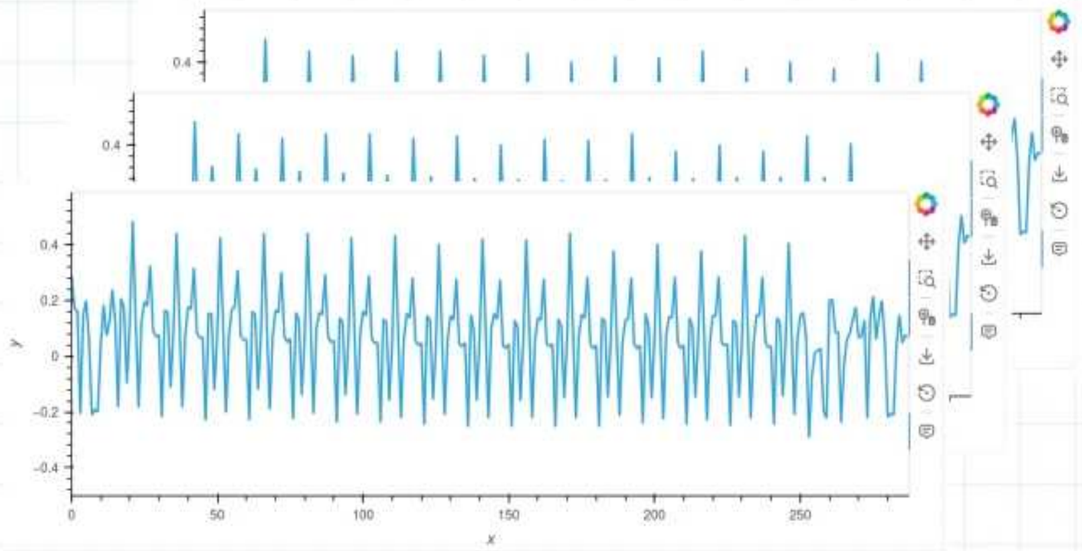
→ `[0x7C, 0xA2, ...]`

https://lightturtles.github.io

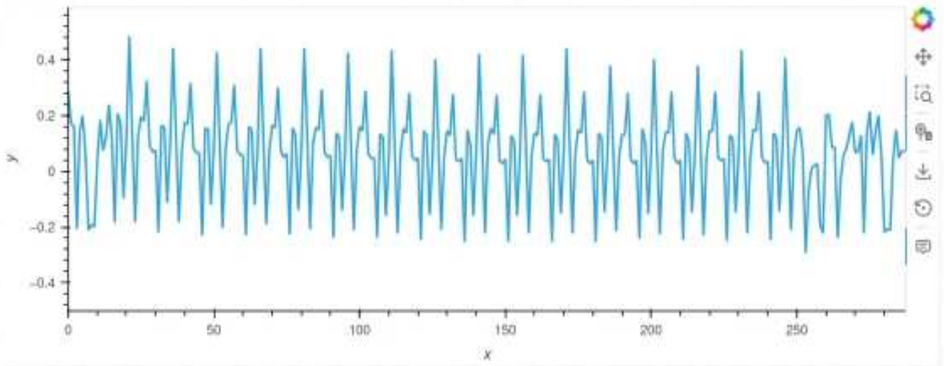
1) Average many traces (reduces the noise in each trace)

Differential Power Analysis

1) Average many traces (reduces the noise in each trace)

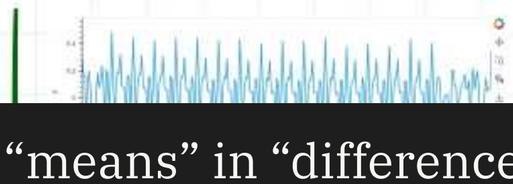


Avg "0x1A" trace

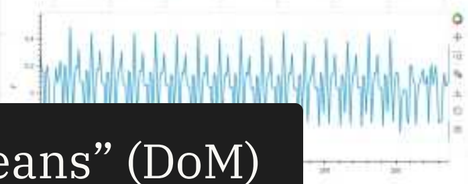


2) Subtract the averages of the two traces

Avg "0x1A" trace



Avg "0x1B" trace



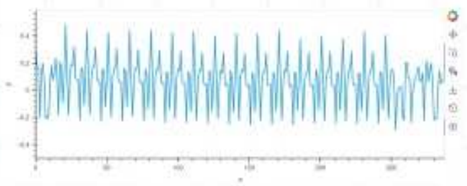
The “means” in “difference of means” (DoM)

https://lightturtles.github.io

Differential Power Analysis

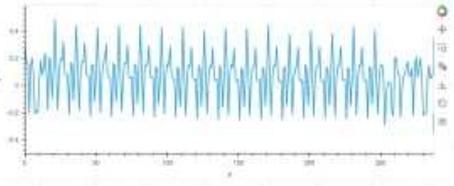
2) Subtract the averages of the two traces

Avg "0x1A" trace



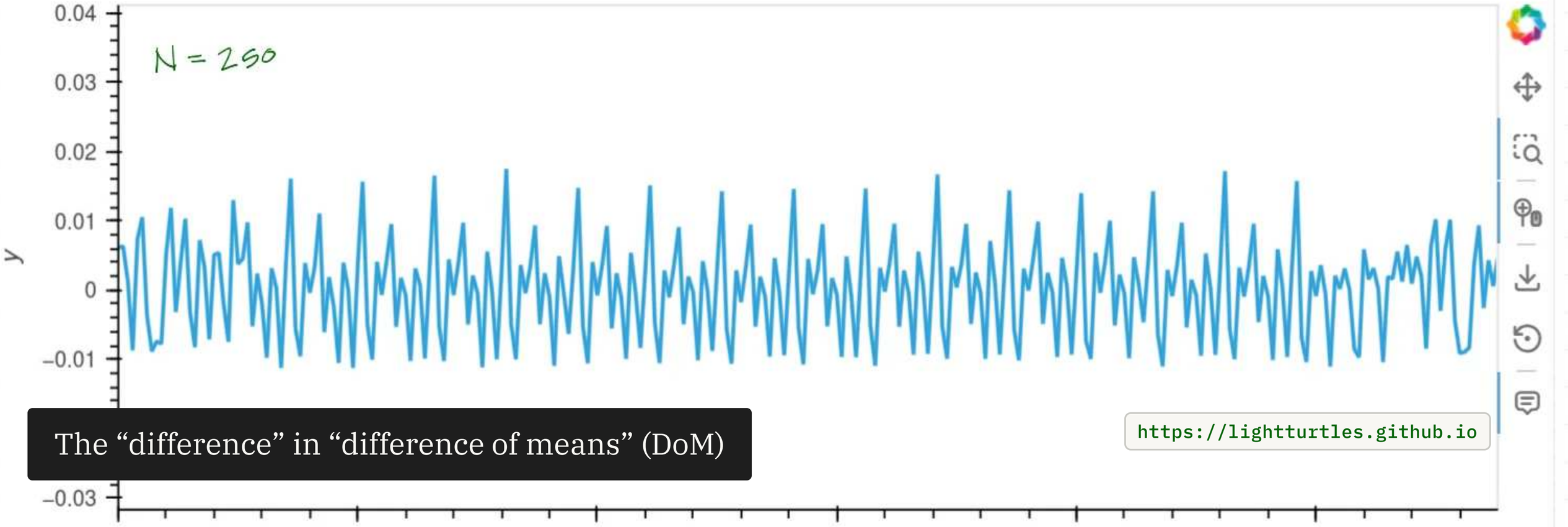
−

Avg "0x1B" trace



=

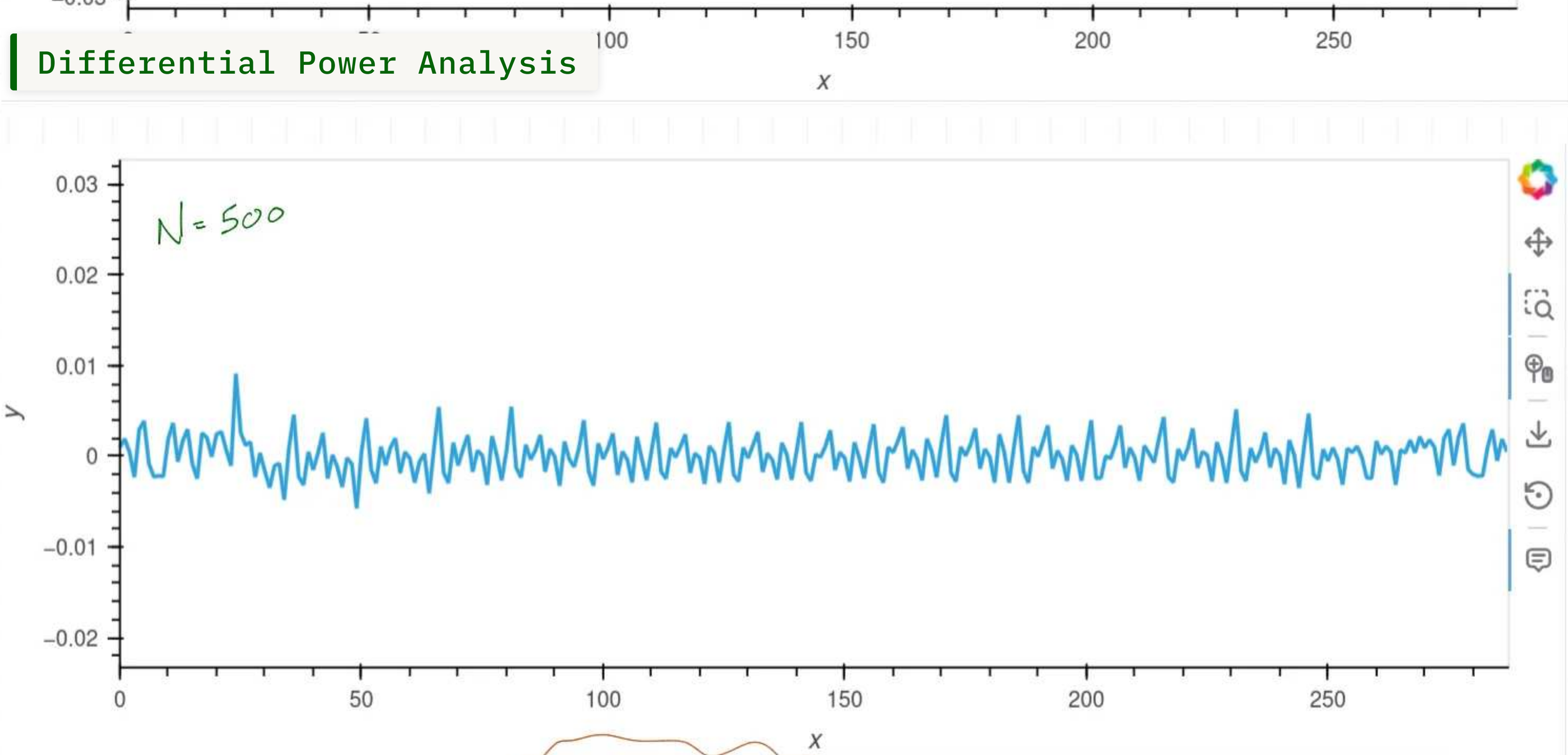
Diff
Mea



The “difference” in “difference of means” (DoM)

https://lightturtles.github.io

Differential Power Analysis

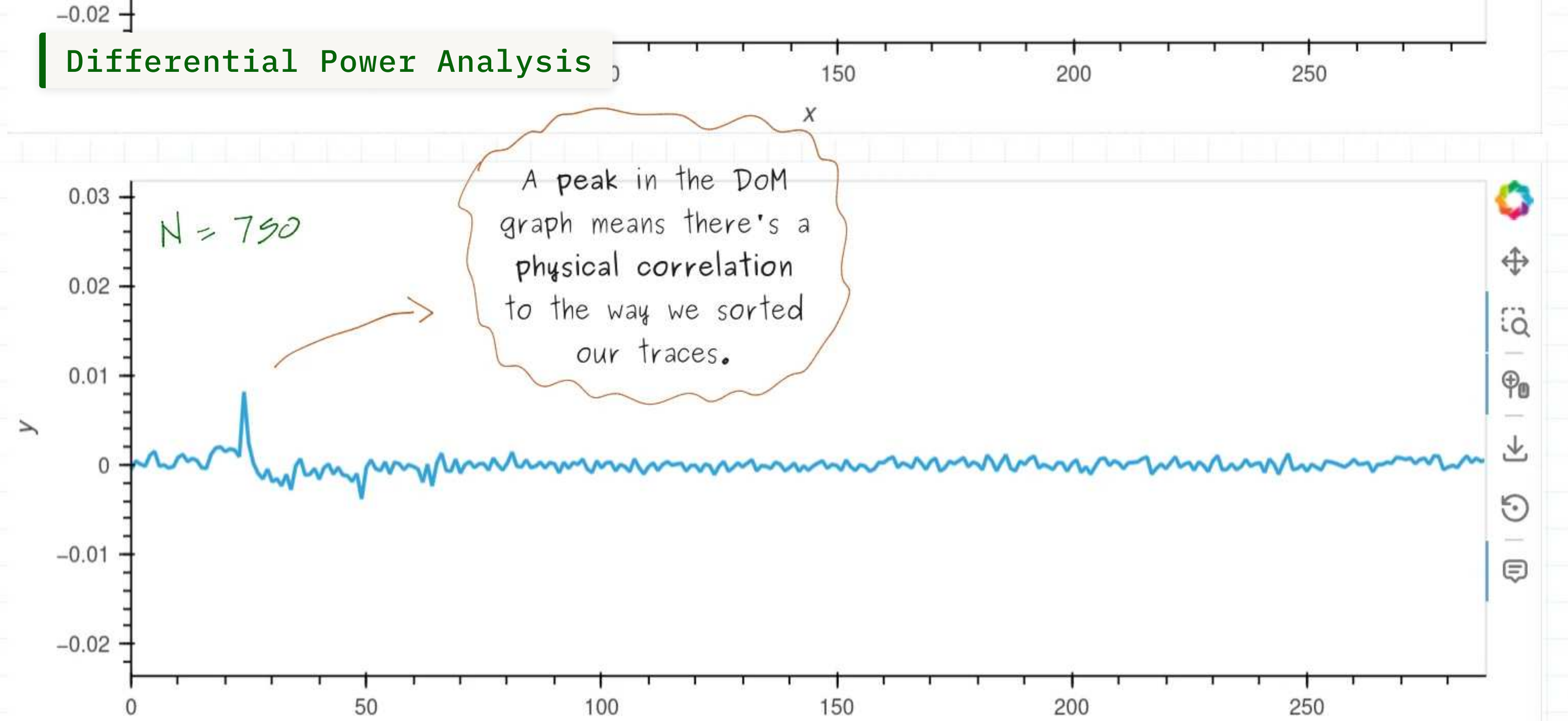


Still no peak after averaging 500 traces...



https://lightturtles.github.io

Differential Power Analysis



We can see the effect of one bit in the power trace after averaging 750 traces!

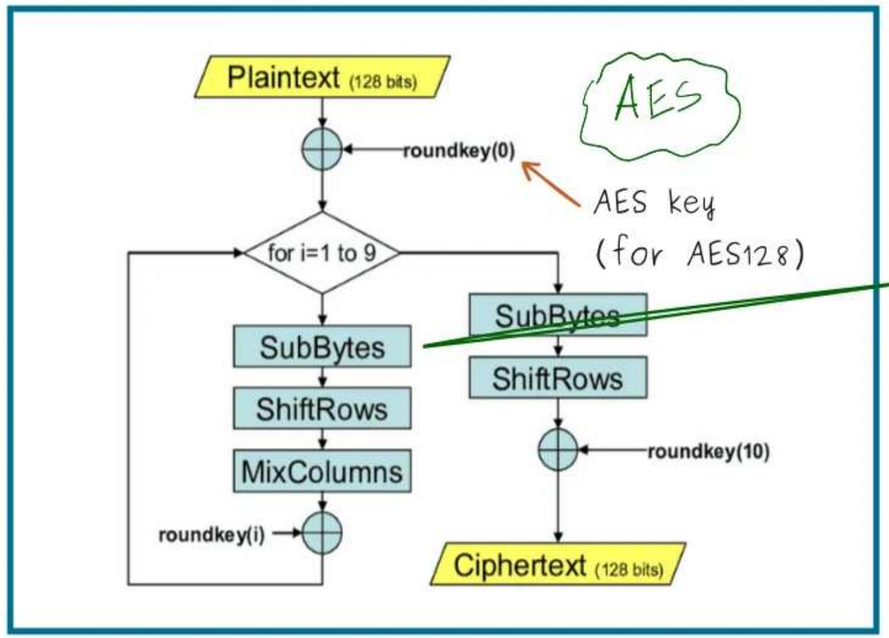
https://lightturtles.github.io

- Corollary: Exact same operation is occurring

Differential Power Analysis

(... number of traces to reduce noise

Difference of Means (DoM) → We can use this to attack AES!!



(R06)

AES S-box

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

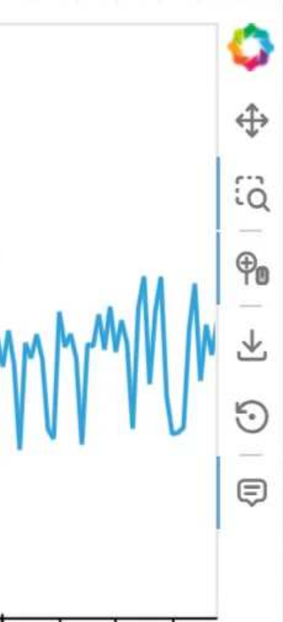
(R05)

1) Collect many (1000s) of traces of device performing AES encryption



[0x34, 0xA0, ...]





AES

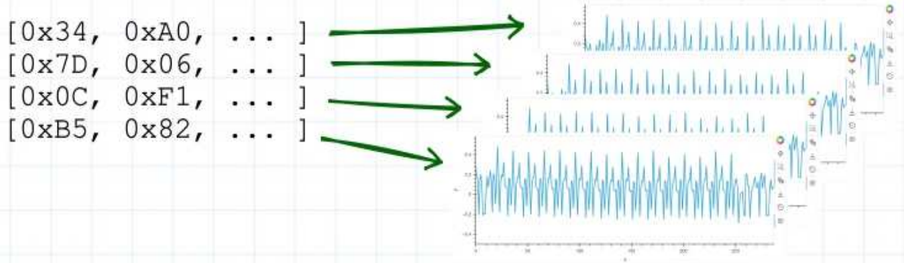
https://lightturtles.github.io



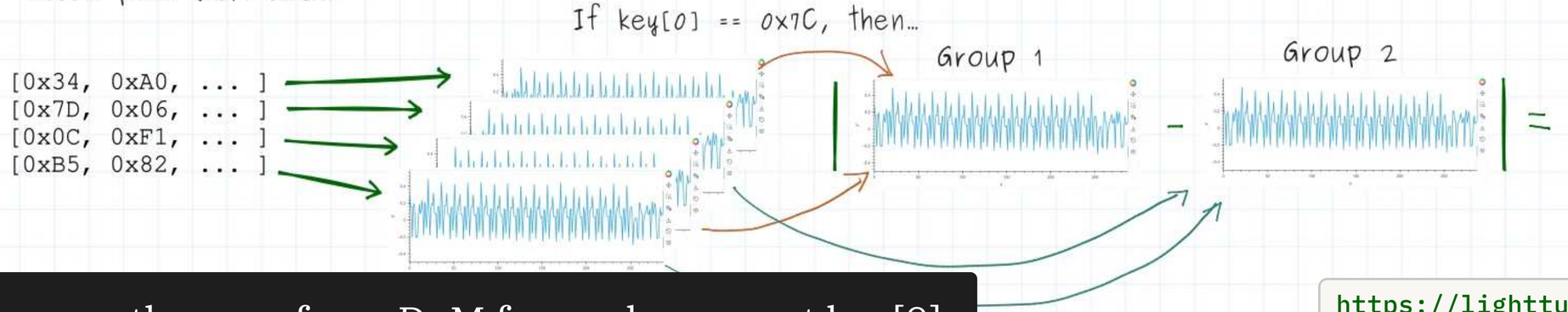
Differential Power Analysis

1) Collect many (1000s) of traces of device performing AES encryption





2) 256 times (for each possible value of key[0]):
- Sort traces based on sbox[in[0] ^ key_guess] (either LSB or HW)
- Calculate DoM
- Store peak DoM value

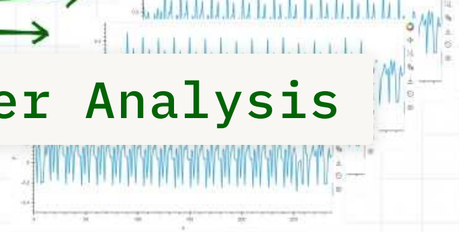


Collect traces, then perform DoM for each guess at key[0]

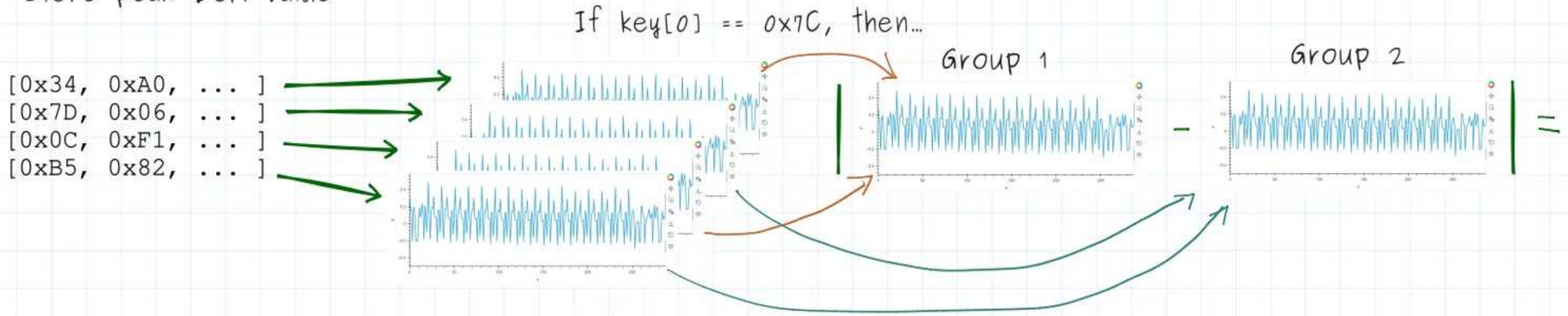
4) Repeat steps 2 and 3 for each other byte of the 16-byte AES key

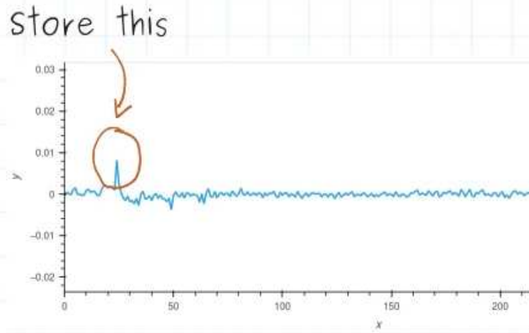
https://lightturtles.github.io

Differential Power Analysis



2) 256 times (for each possible value of key[0]):
   - Sort traces based on sbox[in[0] ^ key_guess] (either LSB or HW)
   - Calculate DoM
   - Store peak DoM value





3) The key guess with the highest peak is the likeliest value!
4) Repeat steps 2 and 3 for each other byte of the 16-byte AES key

This is a Differential Power Analysis (DPA)

The guess whose DoM has the highest “peak” is the likeliest value; repeat for key[1:15]

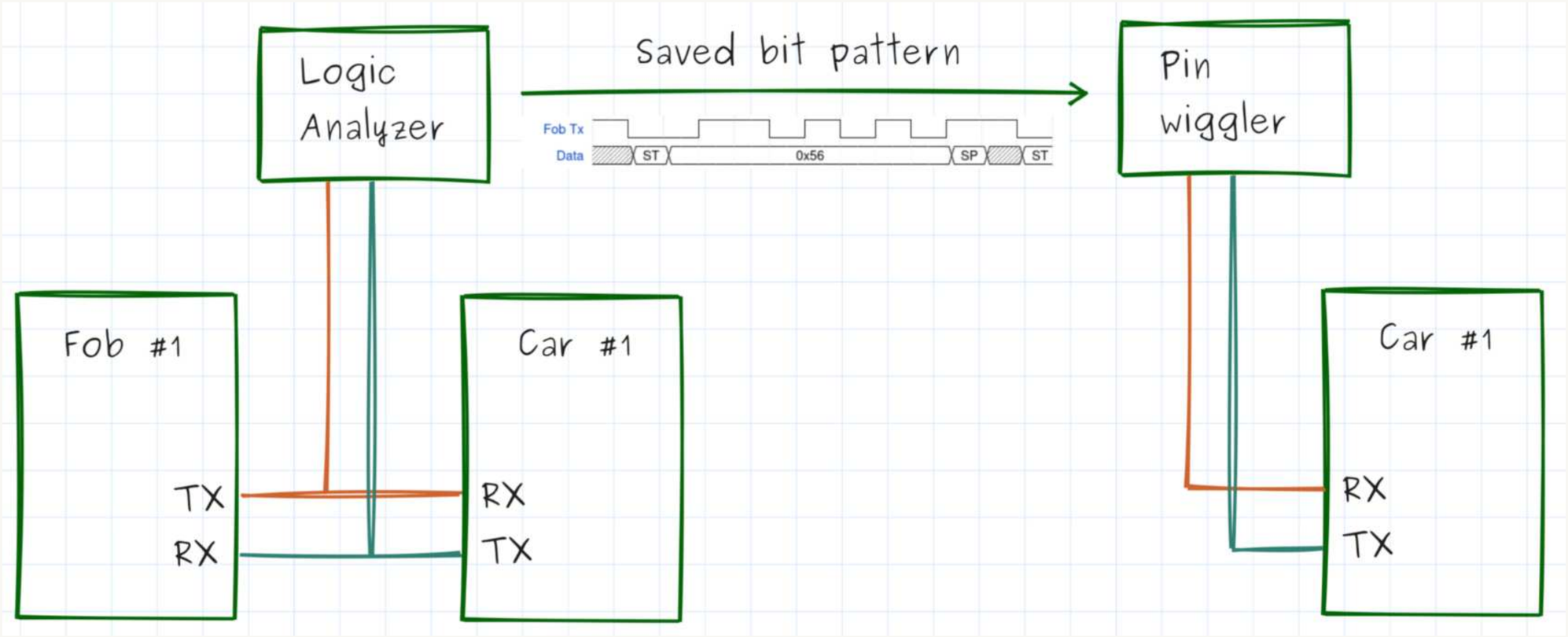
https://lightturtles.github.io

How Hardware Gets Hacked

https://lightturtles.github.io

Article on DigiKey

Based on 2023 MITRE Embedded CTF competition



Fast forward one month: You’re project has failed. What’s the likeliest reason why?

https://lightturtles.github.io

Three things I’d fix



https://lightturtles.github.io

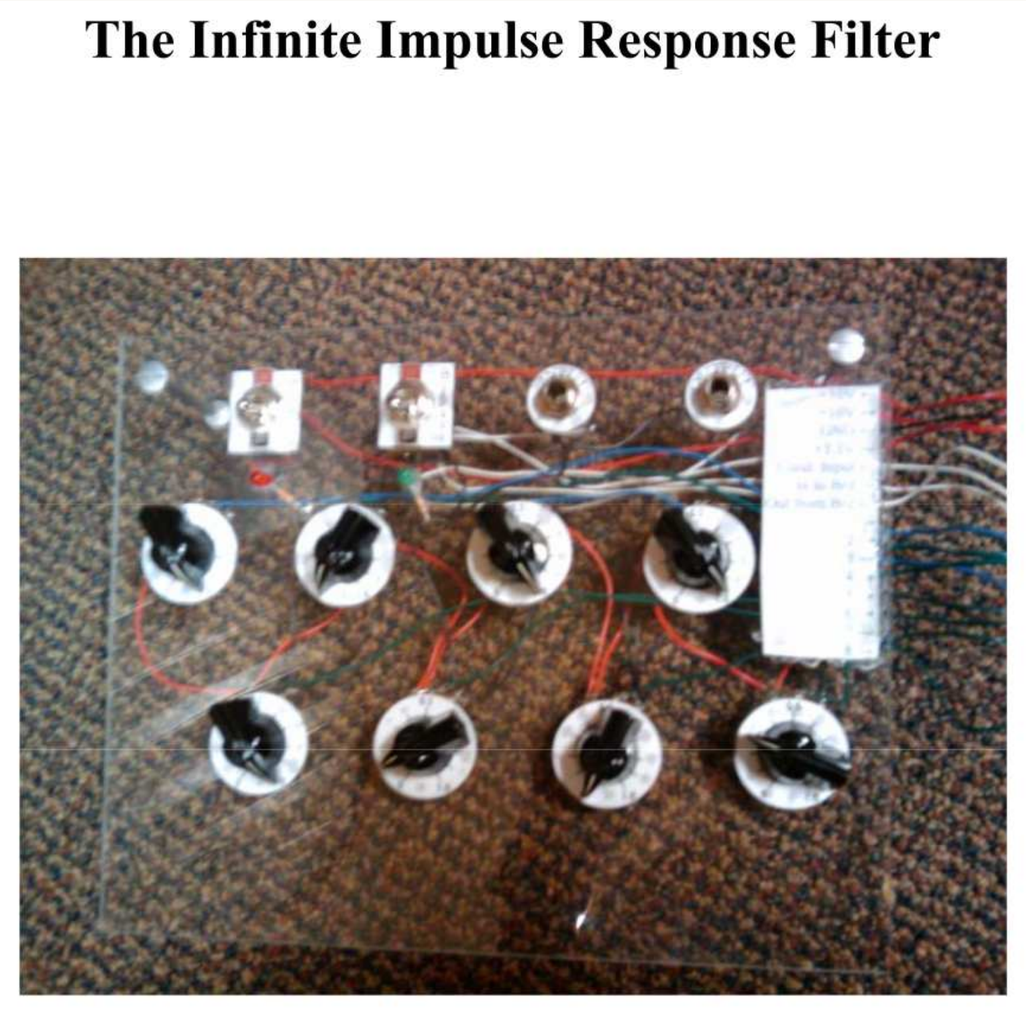
Three things I’d fix



1. Draw a better block diagram

https://lightturtles.github.io

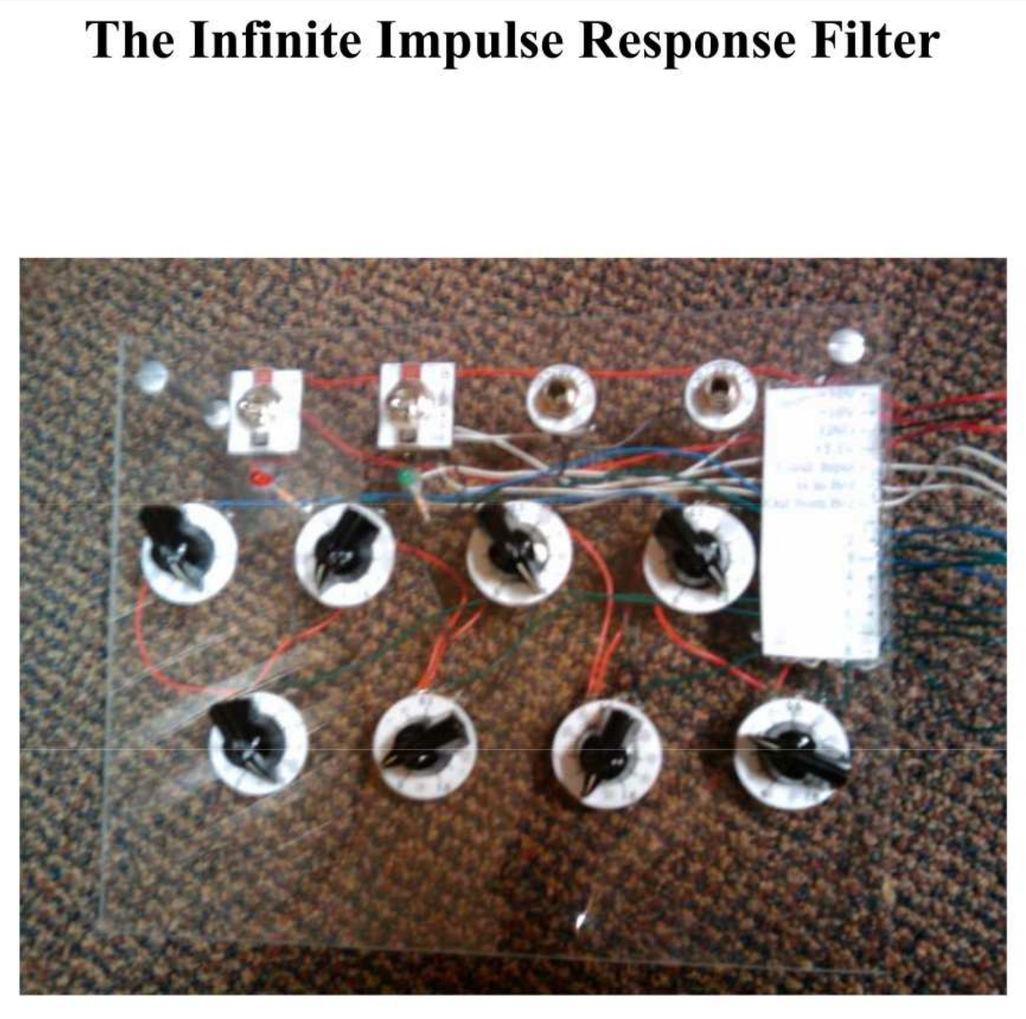
Three things I’d fix



1. Draw a better block diagram
2. Get a working prototype faster

https://lightturtles.github.io

Three things I’d fix



1. Draw a better block diagram
2. Get a working prototype faster
3. Plan for things to go wrong

https://lightturtles.github.io

Draw a better block diagram

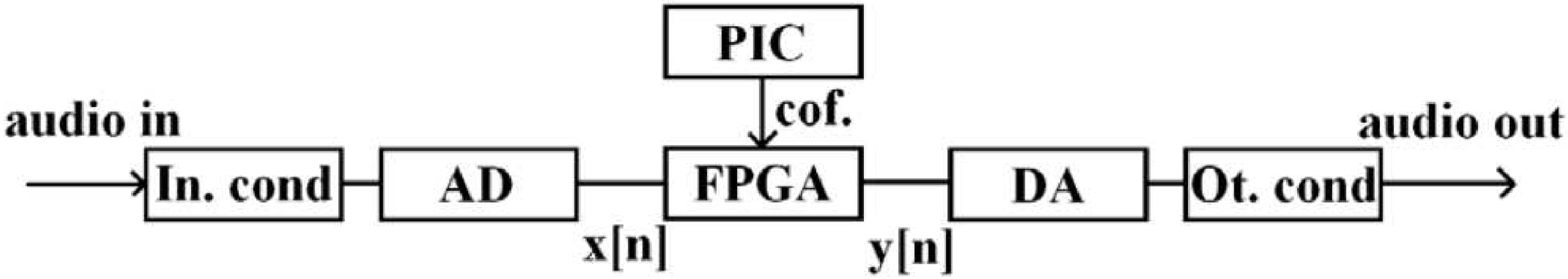


Figure 1: A graphical representation of the IIR filter, including such major components as the input/output conditioning circuitry, the ADC/DAC, and the IIR filter on the PIC/FPGA.

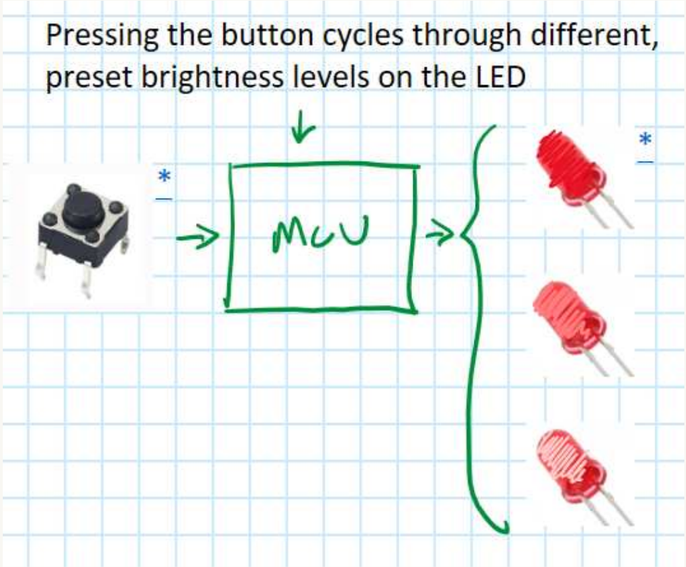
https://lightturtles.github.io

Draw a better block diagram



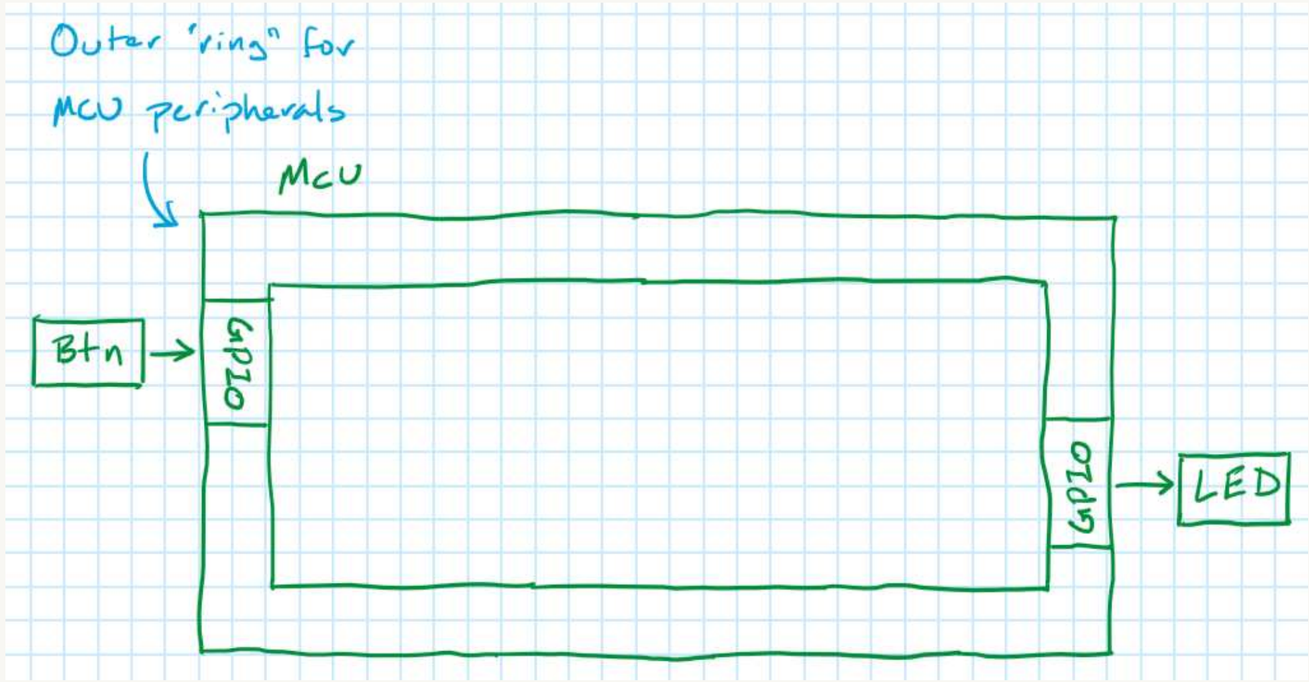
https://lightturtles.github.io

Subdivide until each block seems easy



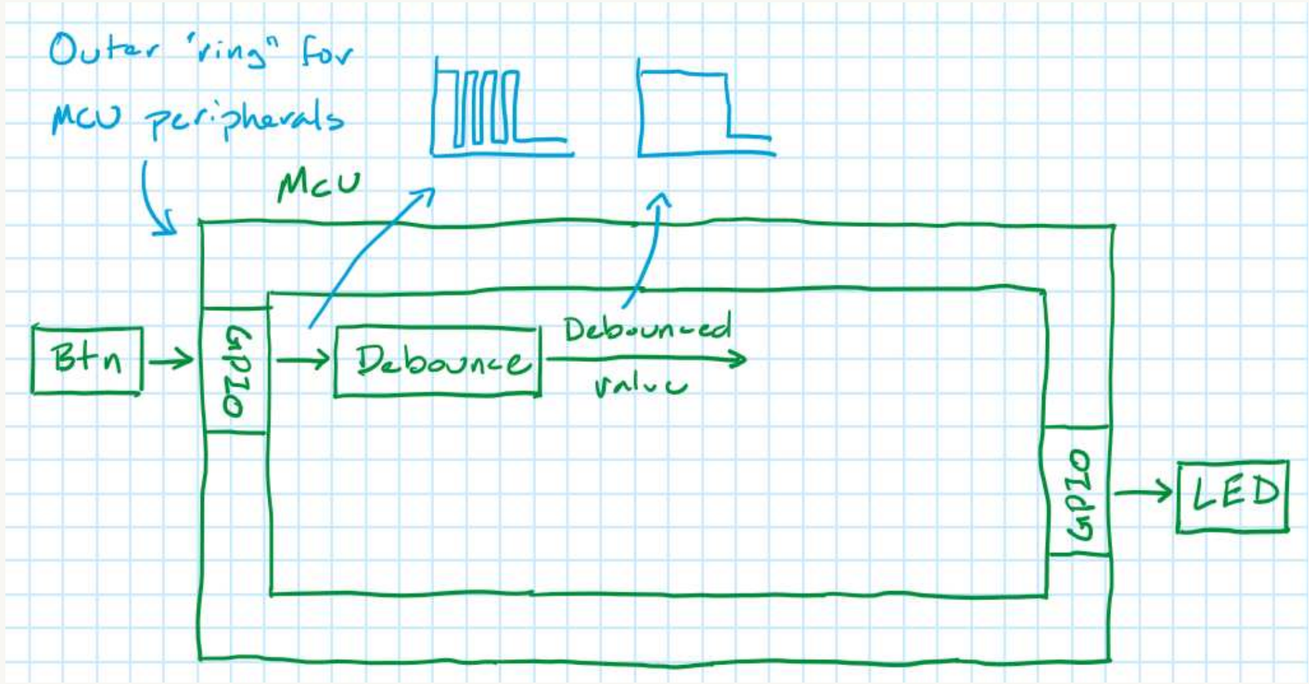
https://lightturtles.github.io

Subdivide until each block seems easy



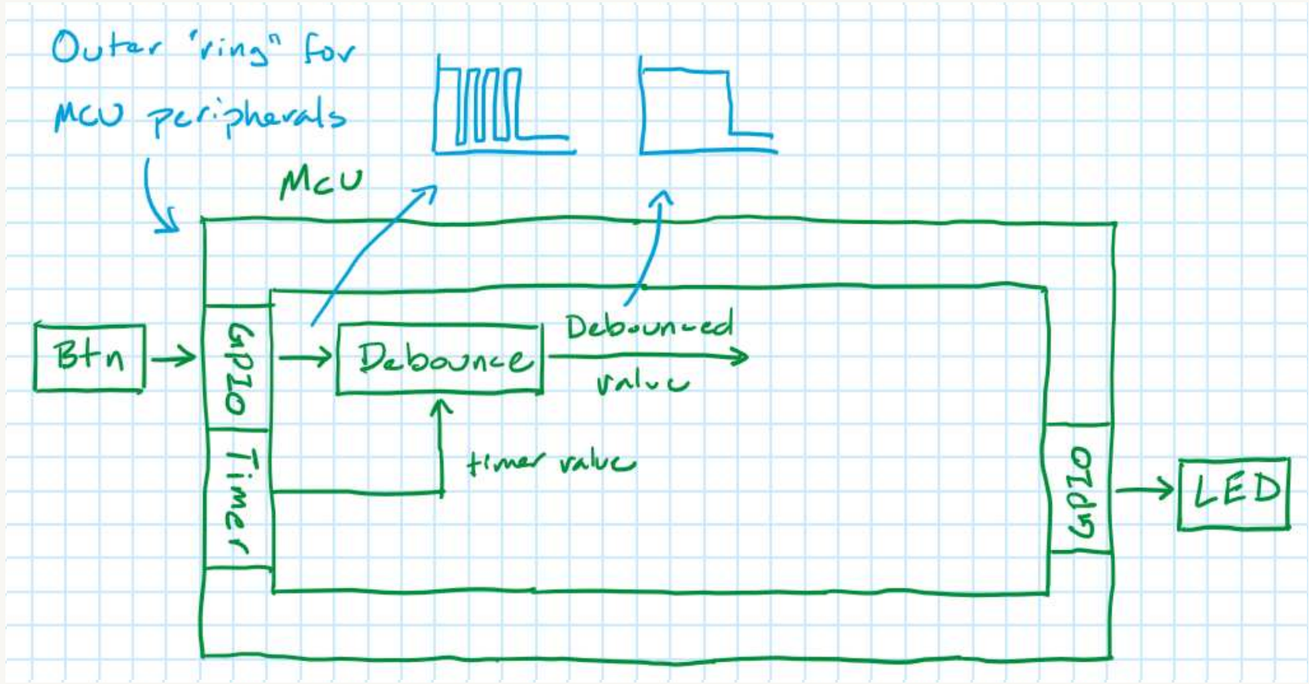
https://lightturtles.github.io

Subdivide until each block seems easy



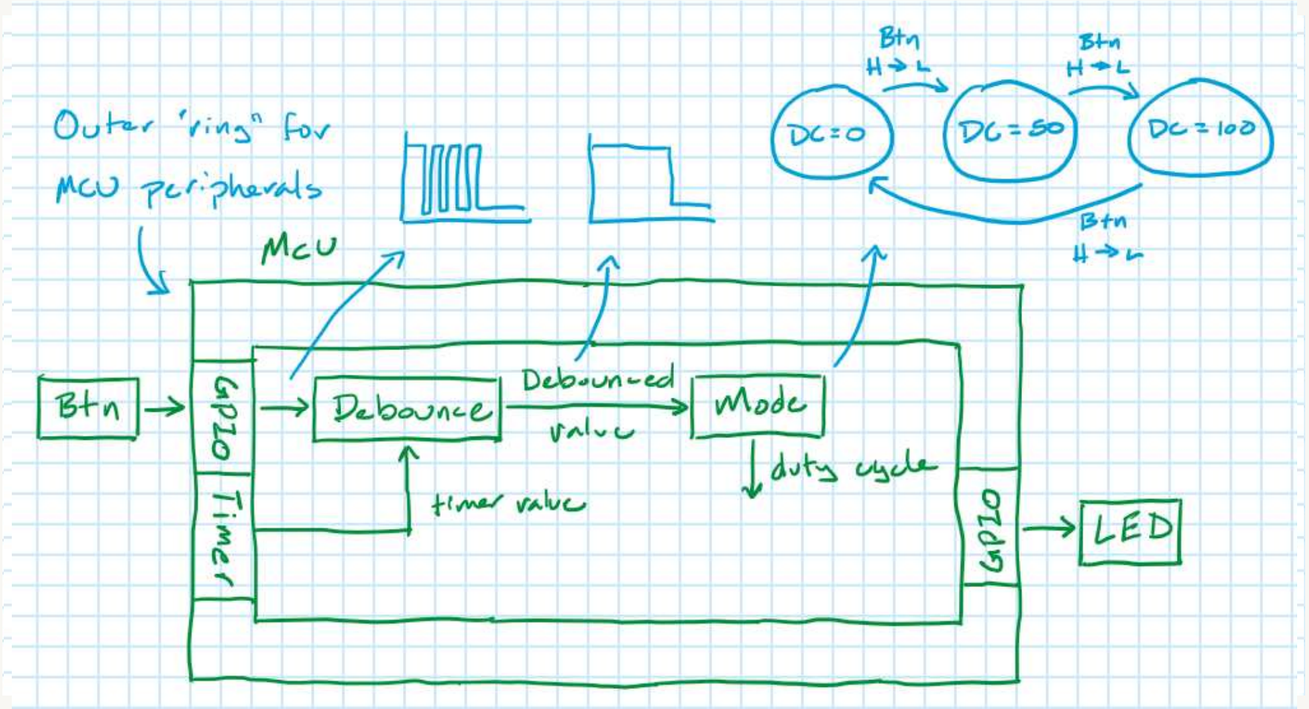
https://lightturtles.github.io

Subdivide until each block seems easy



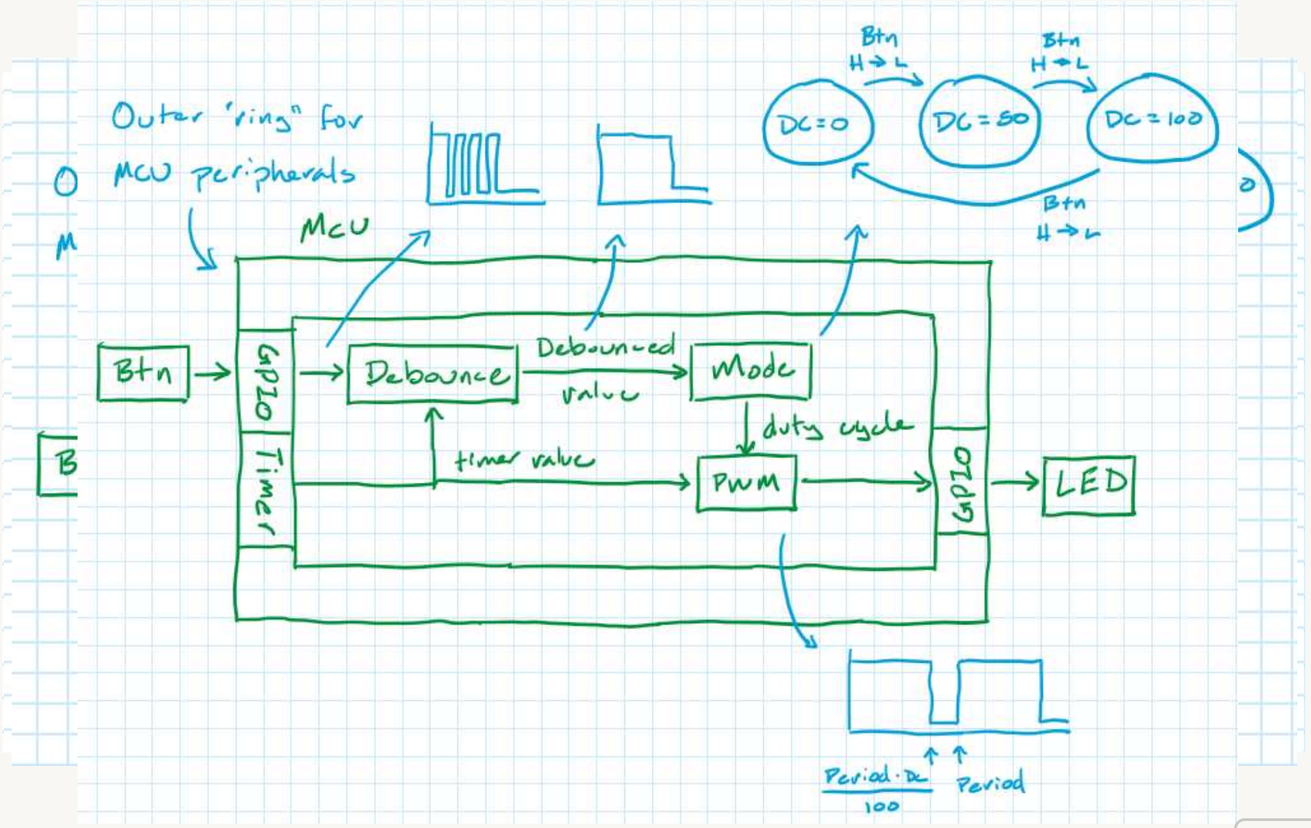
https://lightturtles.github.io

Subdivide until each block seems easy



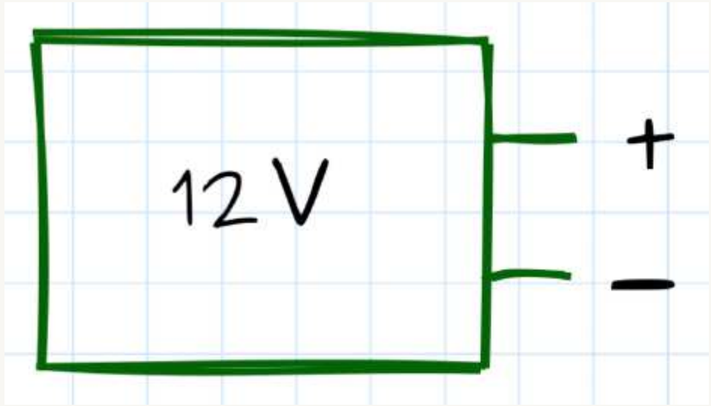
https://lightturtles.github.io

Subdivide until each block seems easy



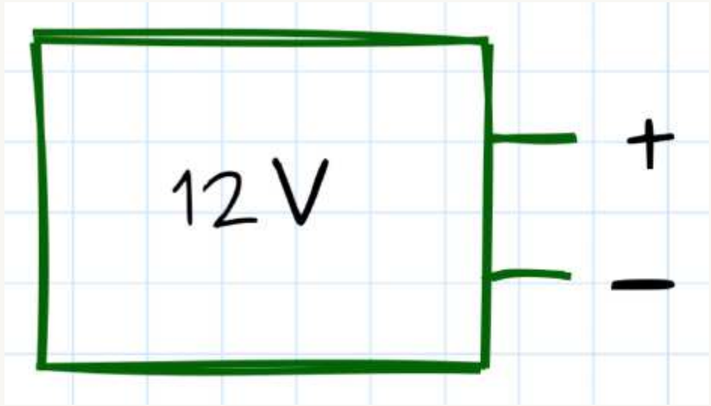
https://lightturtles.github.io

Describe *all* qualities of the input/output



https://lightturtles.github.io

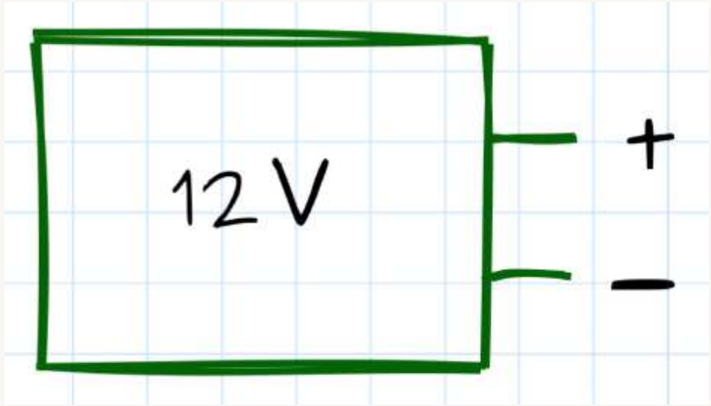
Describe *all* qualities of the input/output



- 12V @ 500 mA

https://lightturtles.github.io

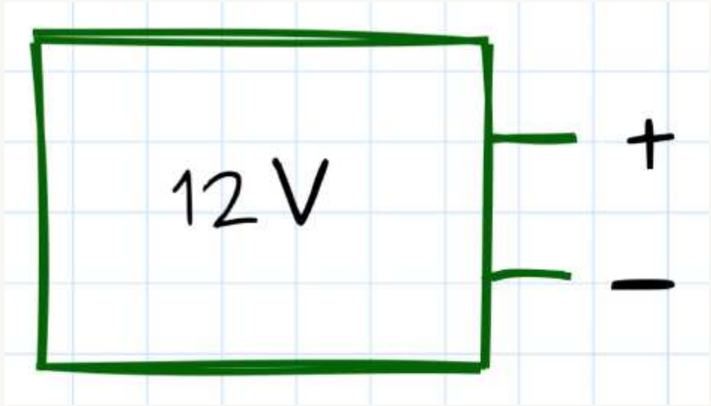
Describe *all* qualities of the input/output



- 12V @ 500 mA
- 12V @ 500 mA with 20° C temperature rise

https://lightturtles.github.io

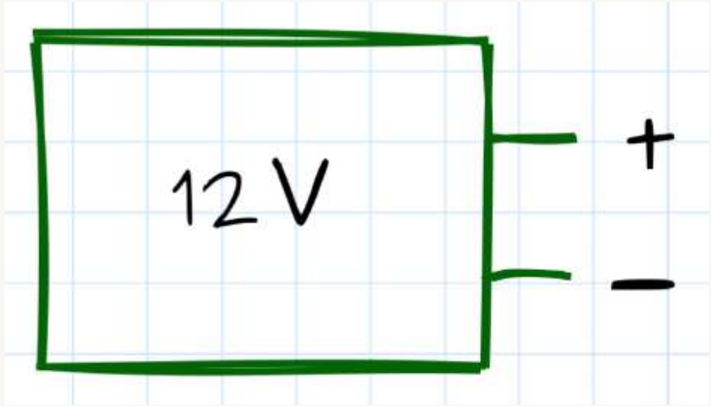
Describe *all* qualities of the input/output



- 12V @ 500 mA
- 12V @ 500 mA with 20° C temperature rise
- 12V @ 500 mA with 20° C temperature rise, assuming 20W heat dissipation to ambient air

https://lightturtles.github.io

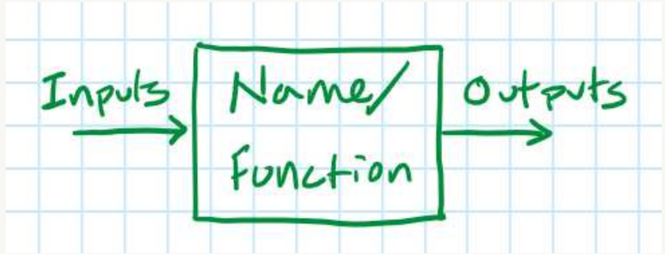
Describe *all* qualities of the input/output



- 12V @ 500 mA
- 12V @ 500 mA with 20° C temperature rise
- 12V @ 500 mA with 20° C temperature rise, assuming 20W heat dissipation to ambient air
- 12V @ 500 mA with 20° C temperature rise, assuming 20W heat dissipation to ambient air, terminating in a JST-PH connector

https://lightturtles.github.io

Draw a better block diagram



- Subdivide your system into blocks until each one seems easy to implement
- For each input/output, ask:
  - What is this signal? What are it's physical properties?
  - What does it mean (i.e. what information does it carry)?
  - How does it change over time?
  - What assumptions must be true for this signal to be valid?
  - What side effects does it have (e.g. heat, EMI, etc)?

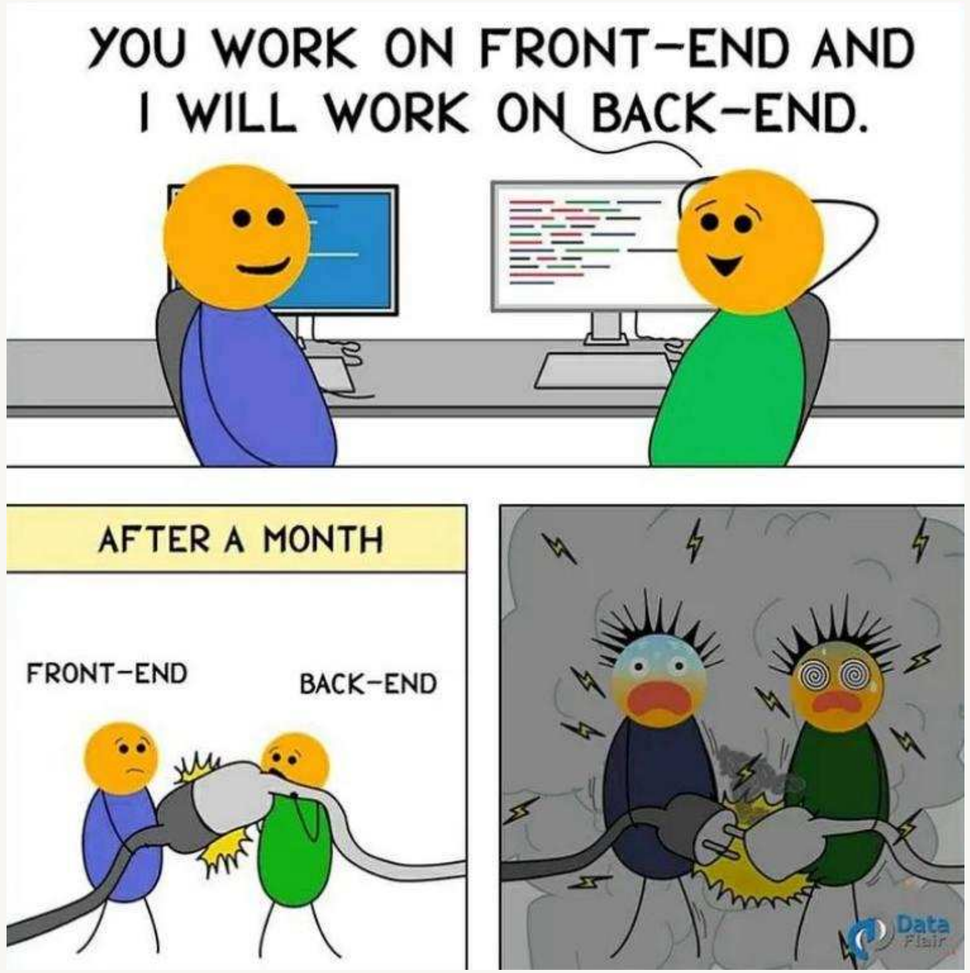
https://lightturtles.github.io

“If you can’t draw it, you don’t understand it.” -Nathan Jones



https://lightturtles.github.io

Get a working prototype faster

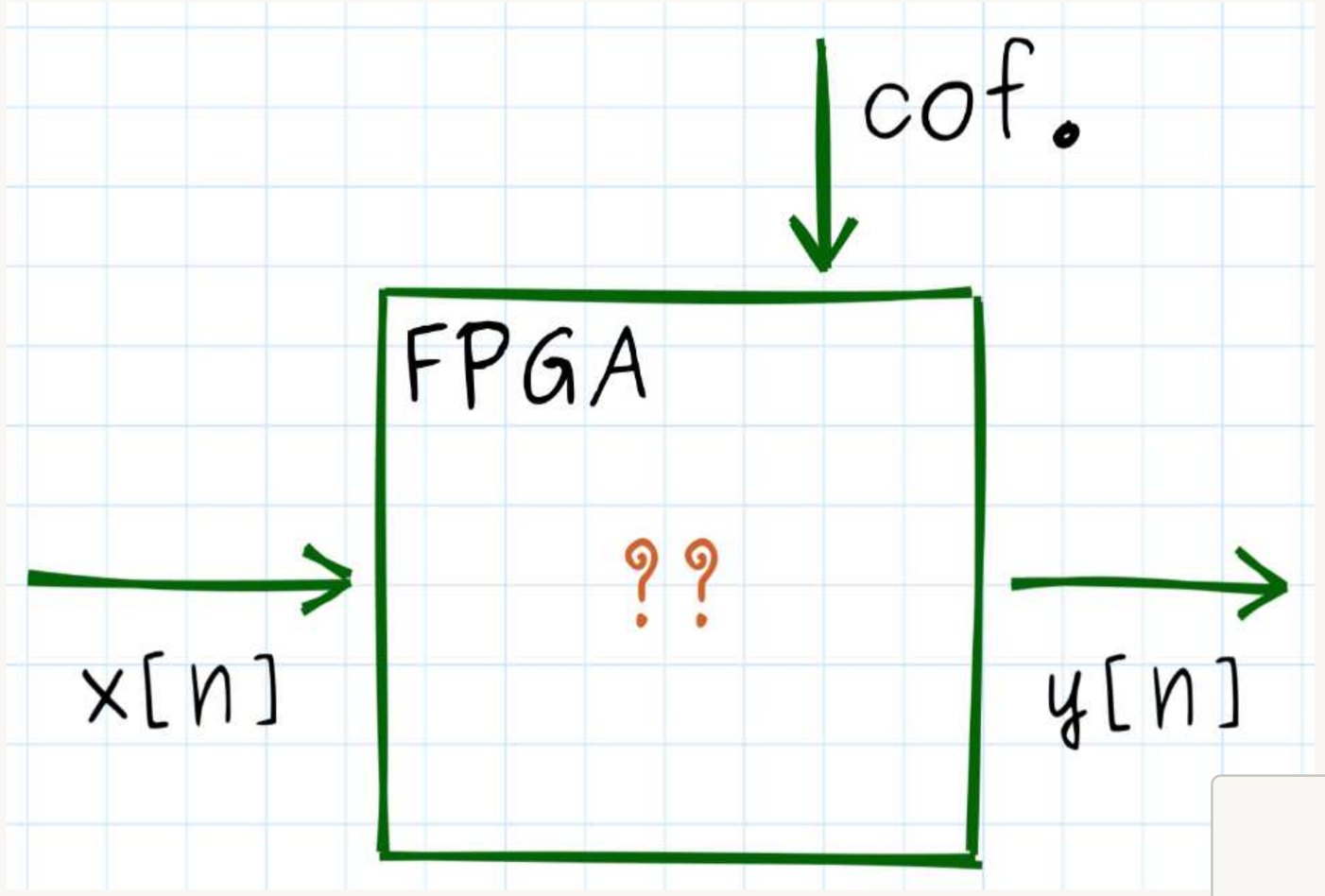


https://lightturtles.github.io

Gall’s Law: “A complex system that works is invariably found to have evolved from a simple system that worked.” - John Gall

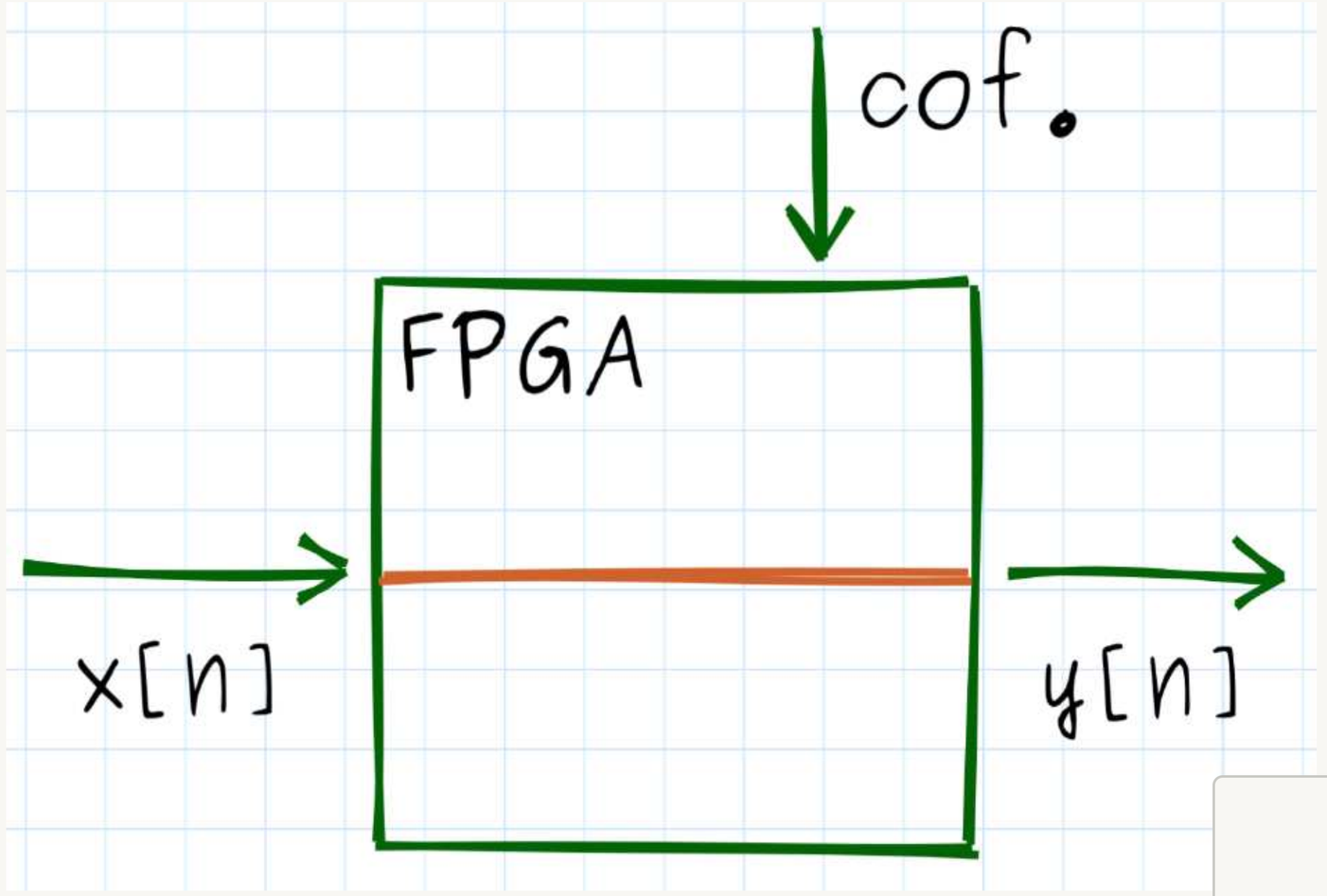
https://lightturtles.github.io

What’s the “simple system” for the FPGA?



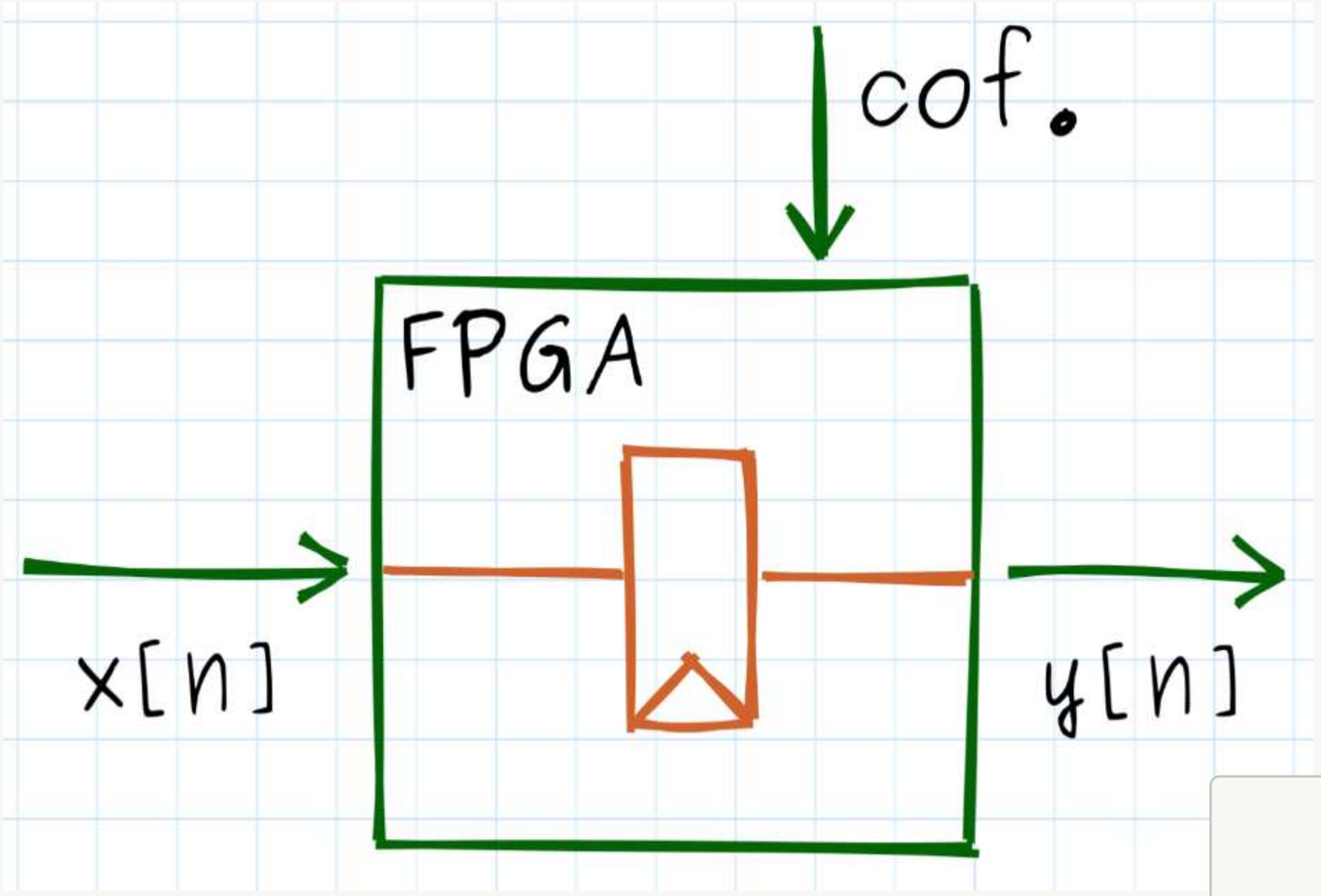
https://lightturtles.github.io

What's the "simple system" for the FPGA?



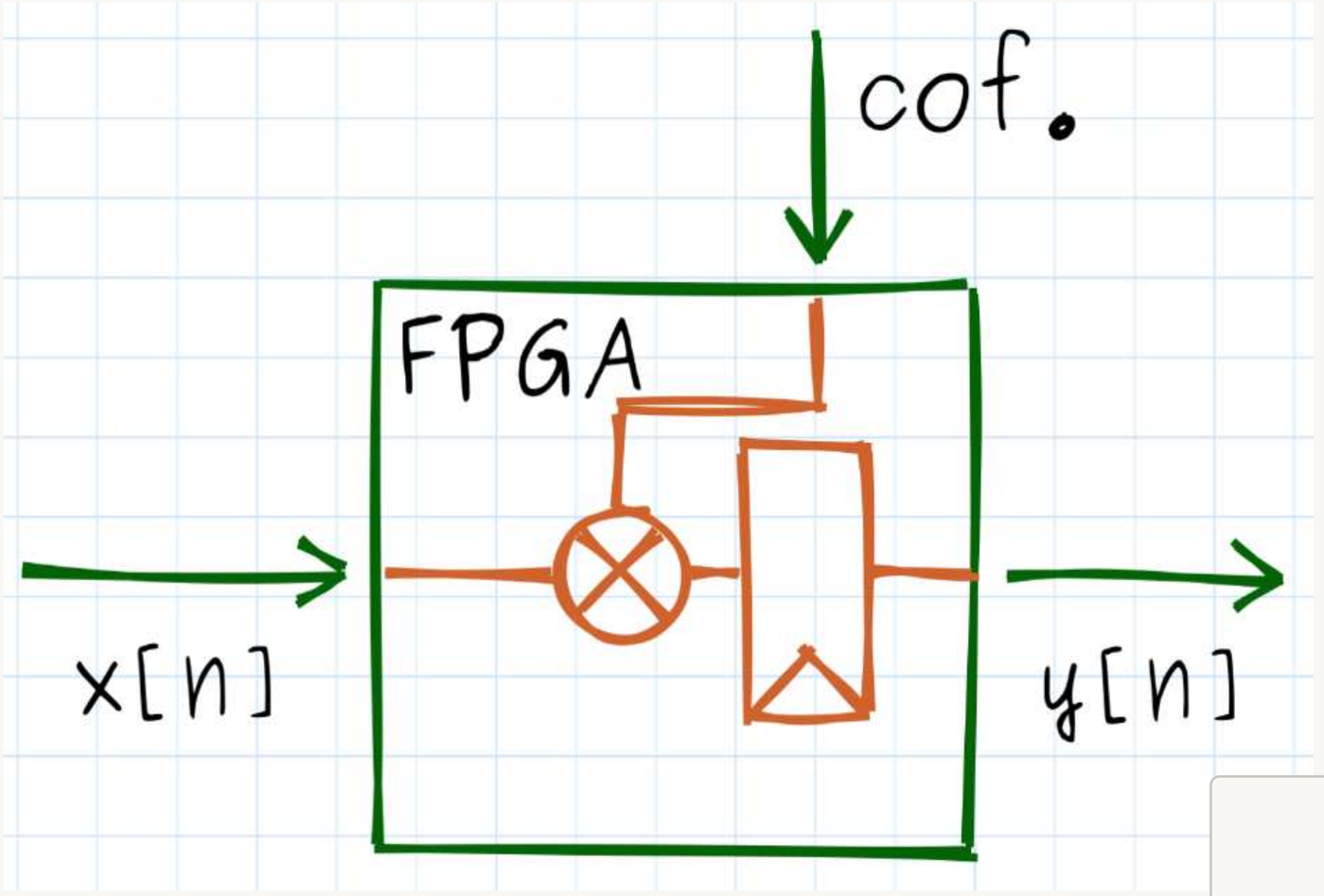
https://lightturtles.github.io

What's the "simple system" for the FPGA?



https://lightturtles.github.io

What's the "simple system" for the FPGA?



https://lightturtles.github.io

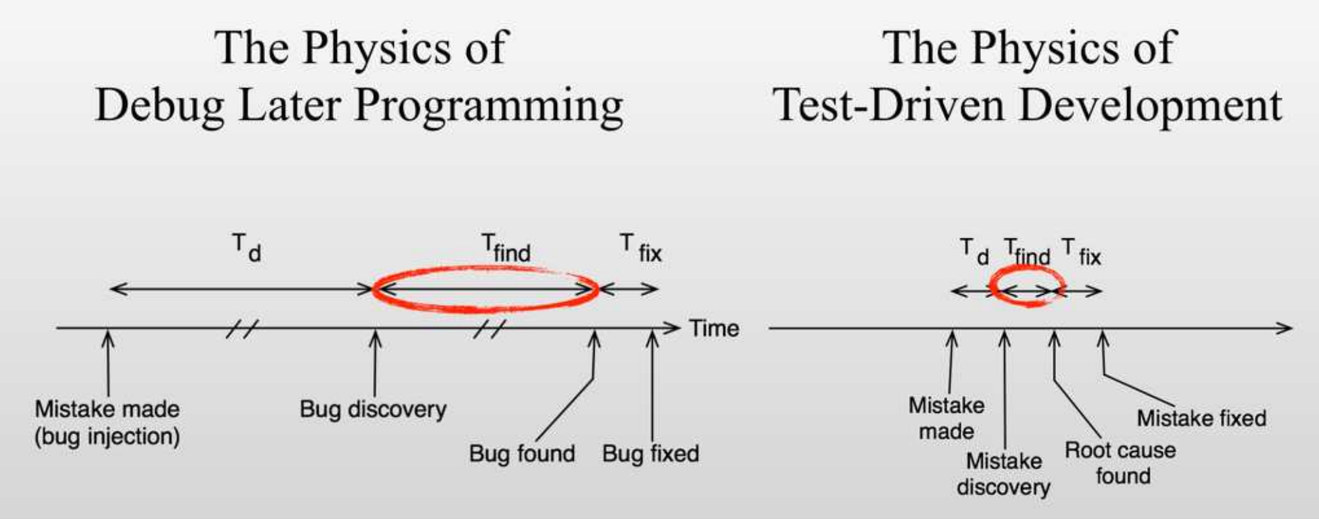
“A good **SCIENTIST** is a person with original ideas. A good **ENGINEER** is a person who makes a design that works with as few original ideas as possible.” -Freeman Dyson



https://lightturtles.github.io

Plan for things to go wrong

- Modularize with the **block diagram** and start with the **simplest working system** (and be able to revert back to it if needed!)
- Test early and often
- Instrument everything
  - `printf`
  - Toggling digital IO
  - Error LEDs
- Write down what you do
- Take breaks to clear your head



https://lightturtles.github.io

Ask me anything!

https://lightturtles.github.io



- Master's of Computer Engineering from NC STATE UNIVERSITY
- 15 years in the U.S.ARMY as a Military Intelligence Officer
  - 5 of those at West Point
- Owner/operator LightTurtles, LLC Embedded Systems Design & Education (lightturtles.github.io)
- Areas of expertise
  - Firmware architecture
  - Hardware security
- Married 14 years with 2 kids

Stay in touch!

